VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 8: Interfaces and Modports

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 8 Overview 4

Part 1: Interfaces 6
- 1.1 The Problem: Connecting Blocks Port by Port 6
- 1.2 What Is an Interface? 7
- 1.3 Interface Declaration, Signals and Instances 10
- 1.4 Interface Ports and Parameters 13
- 1.5 Connecting an Interface to a DUT 14
- 1.6 Clock and Reset Generation Inside Interfaces 15
- 1.7 Adding Values to Interface Signals 18
- 1.8 Tasks, Functions and Assertions Inside Interfaces 19

Part 2: Modports and Interface Arrays 22
- 2.1 What Is a Modport and Why Is It Required? 22
- 2.2 Input, Output and Inout Directions in a Modport 23
- 2.3 Different Modports for the DUT and the Testbench 24
- 2.4 Selecting the Modport: Module Header or Instance 27
- 2.5 Accessing Interface Signals 28
- 2.6 Tasks and Clocking Blocks in Modports 29
- 2.7 Interface Arrays 30
- 2.8 Generic Interface Ports 33
- 2.9 Summary of Part 2 33

Part 3: Virtual Interfaces 34
- 3.1 The Problem: A Class Cannot Reach a Physical Interface 34
- 3.2 What Is a Virtual Interface? 35
- 3.3 Virtual Interface Declaration 37
- 3.4 Assigning a Physical Interface to a Virtual Interface 39
- 3.5 Accessing Interface Signals from Classes 40
- 3.6 Passing Interface Handles to Classes 43
- 3.7 The Null Virtual Interface Error 44
- 3.8 Arrays of Virtual Interfaces 47
- 3.9 Virtual Interfaces with Modports and Parameters 49
- 3.10 Summary of Part 3 50

Part 4: Clocking Blocks 52
- 4.1 The Problem: Races Between the Testbench and the DUT 52
- 4.2 How the Simulator Orders Events in One Time Step 54
- 4.3 What Is a Clocking Block? 55
- 4.4 Input and Output Skews 57
- 4.5 Sampling and Driving Through a Clocking Block 59
- 4.6 Race Prevention with a Clocking Block 61
- 4.7 Cycle Delays (##N) and Default Clocking 63

Part 5: Putting It Together 66
- 5.1 The Final ALU Interface 66
- 5.2 Complete Example: Classes Driving and Monitoring the ALU 67
- 5.3 Bidirectional Signals in an Interface 72
- 5.4 Program Blocks 75

5.5 Interface, Modport, Clocking Block and Virtual Interface Compared 76

Part 6: Module 8 Revision and Practice 78

6.1 Revision Summary 78
6.2 Interview Questions and Answers 80
6.3 Output-Prediction Questions 82
6.4 Find-the-Error and Debugging Questions 85
6.5 Coding Exercises with Solutions 89
6.6 Concepts You Must Understand Before Moving Forward 96
6.7 What Comes Next 97

Module 8 Overview

This module explains how a testbench is physically connected to the design. Modules 4 to 7 built the software side of a testbench: classes, random transactions, mailboxes, parallel threads. None of that can touch a DUT pin yet. The constructs in this module are the bridge between class-based code and the signals of the DUT.

What this module covers

Part	Topic	Question it answers
1	Interfaces	How do I bundle the signals of a bus into one reusable unit?
2	Modports and interface arrays	How do I say who drives which signal, and how do I handle several copies of a bus?
3	Virtual interfaces	How does a class, which is created at run time, reach signals that were created at elaboration time?
4	Clocking blocks	When exactly should the testbench sample and drive, so that it never races with the DUT?
5	Putting it together	The final ALU interface, classes that drive and monitor the ALU, bidirectional signals, program blocks
6	Revision and practice	Summary, interview questions, output prediction, debugging, exercises

What you need from earlier modules

From	You must already know
Module 1	`logic` versus `wire`, the single-driver rule for variables, four-state values, the ALU design (`alu_pkg`, `alu`)
Module 3	Tasks, functions, `automatic` lifetime, argument directions
Module 4	Classes, handles, `null`, constructors, the `this` keyword
Module 7	`fork` ... `join_none`, events, blocking and nonblocking behaviour of processes

Naming used in this module

Name	Meaning
`alu_if`	The interface for the ALU. Its final form (with clocking blocks and modports) is used unchanged in Module 9
`vif`	A virtual interface handle inside a class. Always this name
`drv_cb`, `mon_cb`	The clocking blocks used by the driver and by the monitor
`DUT`, `DRV`, `MON`	The modports of `alu_if`
`cnt_if`, `counter`	A 4-bit counter and its interface: a small design used to demonstrate one idea at a time
`tb_<topic>`	A small demonstration testbench

How the examples are organised. Small examples are complete in one file. Several examples share a design, which is given once in its own file; the text before each example lists the files to compile and their order. The ALU examples need `alu_pkg.sv` and `alu.sv` from `examples/module-01/` to be compiled first.

> **Note:** As in the other modules, the "Expected output" blocks were derived by hand from the IEEE 1800-2017 rules by tracing every clock edge; they were not captured from a simulator. All stimulus in this module is set by hand (no randomization), so each output is fully determined. Where an example is deliberately racy, the text says that the result depends on the simulator instead of printing one result as fact.

Part 1: Interfaces

1.1 The Problem: Connecting Blocks Port by Port

In plain English

A chip is built from blocks that talk to each other over groups of wires. A group of wires that belong together, with rules about how they are used, is called a **bus** or a **protocol interface**: an AXI bus, an APB bus, an SPI link, or the handful of signals between our testbench and the ALU.

In Verilog there is no way to say "this is one bus". Every wire of the bus is listed separately, again and again: in the port list of each module that uses it, in the declarations of the parent module, and in every instantiation. A bus with 40 signals connecting three blocks means writing each of those 40 names about ten times.

What this looks like for the ALU

The ALU from Module 1 has eight ports. Here is everything a Verilog-style testbench must write to connect it:

```
module tb_top;
  import alu_pkg::*;

  // 1. Declare every signal once more in the testbench
  logic       clk = 0;
  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  // 2. Connect every signal by name
  alu dut (
    .clk       (clk),
    .rst_n     (rst_n),
    .valid_in  (valid_in),
    .a         (a),
    .b         (b),
    .op        (op),
    .valid_out (valid_out),
    .result    (result)
  );

  // 3. A driver module and a monitor module would each need the same eight ports,
  //    the same eight declarations and the same eight connections again.
endmodule
```

Why this is a real problem

Problem	Consequence
The same list of signals is repeated in many places	Long, repetitive code. For a real bus this is hundreds of lines
Adding or renaming one signal means editing every one of those places	A change that should take one line takes an hour, and one missed place is a bug

Problem	Consequence
Nothing ties the declarations together	One module can declare `addr` as 16 bits and another as 32 bits. The tool gives at most a width warning
A misspelled name in a connection creates an implicit 1-bit wire	The signal is silently left unconnected (Module 1, Section 2.4)
The protocol rules live somewhere else	The checks and helper tasks for the bus cannot be kept with the bus signals
A class cannot have ports	There is no way at all to hand "the bus" to a driver class

The last row is the important one for verification. Even if you accept all the typing, a class-based testbench needs a single object that represents the bus and can be passed around. Verilog has nothing like that.

Interview questions

Q1. What problems do Verilog port connections have for large designs? Every signal of a bus is declared and connected separately in every module that uses it, so the code is long and repetitive, a change to the bus has to be repeated in many files, mismatched widths and misspelled names are easy to introduce, and the protocol's checks and tasks cannot be packaged with the signals.

Q2. Does `.*` solve the problem? Only the typing at the instantiation. The signals must still be declared individually in each module, the port lists are still long, and there is still no single object that represents the bus.

1.2 What Is an Interface?

In plain English

An interface is a named bundle of signals. You declare the bundle once, create one copy of it, and connect that single copy to every block that uses the bus. A block that receives the bundle refers to a signal inside it with a dot, like a struct member: `bus.addr`.

Unlike a struct, the bundle can contain more than data. It can hold the direction rules of the bus, the timing rules for the testbench, helper tasks, and protocol checks.

Technical definition

An `interface` is a SystemVerilog design element, declared between `interface` and `endinterface`, that encapsulates the communication between blocks. It is instantiated like a module and creates its own level of hierarchy. It may contain:

An interface may contain	Purpose	Covered in
Nets and variables	The signals of the bus	1.3
Ports and parameters	Signals that come from outside (usually clock), configurable widths	1.4
`modport` declarations	Direction of each signal as seen by each kind of user	Part 2
`clocking` blocks	Synchronous sampling and driving rules for the testbench	Part 4
Tasks and functions	Bus operations such as "write one word" (a bus functional model)	1.8

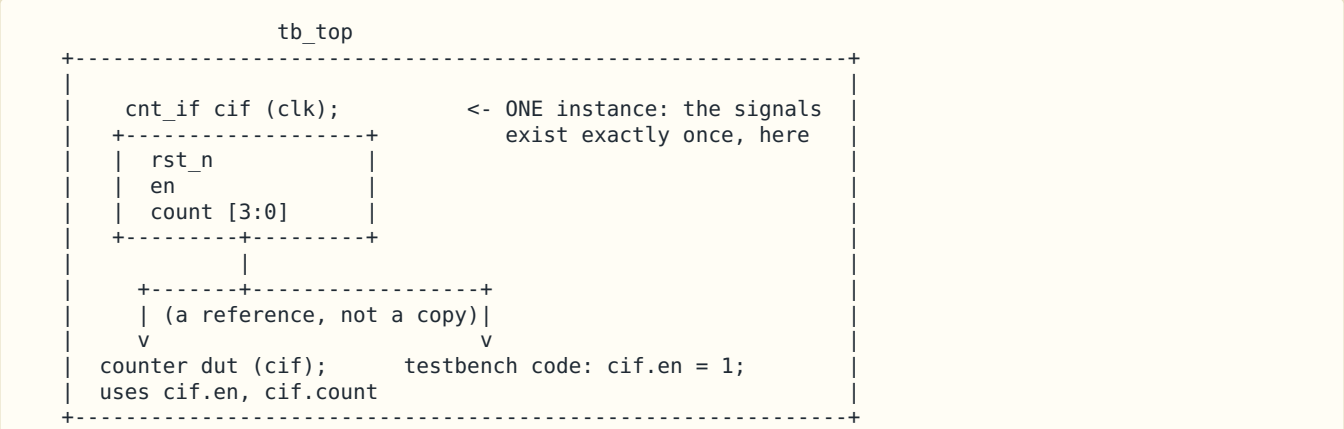
An interface may contain	Purpose	Covered in
`initial`, `always`, `assign`	Clock generation, tri-state drivers, simple glue logic	1.6, 5.3
Assertions and covergroups	Protocol checks and coverage that travel with the bus	1.8, Module 10
Instances of other interfaces	Building a large bus out of smaller ones	

An interface may **not** contain an instance of a module.

Why it is needed

Need	How the interface meets it
Declare the bus once	One `interface` declaration; every user refers to it
Change the bus in one place	Add the signal to the interface. Blocks that do not use it need no change
Consistent widths and types	There is one declaration, so there is nothing to mismatch
Fewer connection errors	One connection per block instead of one per signal
Keep protocol knowledge with the signals	Tasks, assertions and coverage are written inside the interface
Give class-based code access to the signals	A class holds a *virtual interface* handle that refers to the interface instance (Part 3)

How it works internally



- An interface instance is a scope in the design hierarchy, exactly like a module instance. Its full name here is `tb_top.cif`, and the counter value is `tb_top.cif.count`.
- The signals are created once, inside that instance, when the design is **elaborated** (before simulation starts). They exist for the whole simulation. This is called *static*.
- Passing the interface to a module through an interface port does not copy anything. The port is a reference to the instance. When the DUT writes `cif.count` and the testbench reads `cif.count`, they are using the same variable.

Interface compared with module and struct

Property	`module`	`interface`	`struct`
Purpose	Describes behaviour (logic)	Describes connections between blocks	Groups data values
Instantiated in the hierarchy	Yes	Yes	No, it is a data type
Can contain signals	Yes	Yes	Members only
Can contain `always`, `initial`, `assign`	Yes	Yes	No
Can contain tasks, functions, assertions	Yes	Yes	No
Can contain module instances	Yes	**No**	No
Can contain interface instances	Yes	Yes	No
Can be passed through a port as one item	No	Yes (interface port)	Yes (a port of struct type)
Per-user directions	Port directions	`modport`	None: a struct port has one direction for all members
Can be referenced from a class	No	Yes, through a virtual interface	A struct value can be copied into a class
Synthesizable	Yes	Yes, when its contents are	Yes

> **Key point:** A struct port also bundles signals, but every member has the same direction. A bus has signals going both ways (request one way, response the other). That is why a bus is modelled with an interface plus modports and not with a single struct.

Interview questions

Q1. What is an interface in SystemVerilog? A design element that bundles the signals connecting two or more blocks into one named unit. It is instantiated once and connected to each block as a single port. It can also contain modports, clocking blocks, tasks, functions, assertions and procedural code related to the bus.

Q2. What are the advantages of an interface? The bus is declared once; changes are made in one place; connections are shorter and less error-prone; directions (modports), timing (clocking blocks), protocol tasks, assertions and coverage can be kept with the signals; and it is the only way for class-based testbench code to reach DUT signals, through a virtual interface.

Q3. Can an interface contain a module instance? Can a module contain an interface instance? An interface cannot instantiate a module. A module can instantiate an interface, and an interface can instantiate another interface.

Q4. Is an interface synthesizable? Yes. An interface that contains signals, modports and synthesizable logic is accepted by synthesis tools, which flatten it into ordinary ports and wires. Clocking blocks, `initial` blocks, delays and assertions inside it are simulation-only.

Q5. What is the difference between an interface and a struct? A struct is a data type whose members all travel in one direction through a port. An interface is a hierarchical instance that can give each signal its own direction per user, and can contain behaviour: tasks, assertions, clocking blocks and procedural blocks.

1.3 Interface Declaration, Signals and Instances

Syntax

```
// Declaration (at the same level as a module, never inside one)
interface <interface_name> [ #(<parameters>) ] [ (<ports>) ];
  <signal declarations>
  <modports, clocking blocks, tasks, functions, assertions ...>
endinterface

// Instantiation (inside a module, like a module instance)
<interface_name> <instance_name> (<port connections>);

// Use as a port of a module: the port "type" is the interface name
module <module_name> (<interface_name> <port_name>);
  ... <port_name>.<signal> ...
endmodule
```

The demonstration design: a counter

The small examples in Parts 1 to 3 use a 4-bit counter. The interface and the design are in one file, which the later examples reuse.

counter_dut.sv

```
// The bundle of signals between the counter and whatever drives it
interface cnt_if (input logic clk);
  logic       rst_n;      // active-low asynchronous reset
  logic       en;         // count enable
  logic [3:0] count;      // counter value
endinterface

// The design: one interface port instead of four separate ports
module counter (cnt_if cif);

  always_ff @(posedge cif.clk or negedge cif.rst_n) begin
    if (!cif.rst_n)  cif.count <= '0;
    else if (cif.en) cif.count <= cif.count + 1'b1;
  end

endmodule
```

Line by line

Line	Explanation
`interface cnt_if (input logic clk);`	Declares an interface named `cnt_if`. It has one port, `clk`, because the clock is generated outside and is shared with other blocks. Section 1.4 explains interface ports.
`logic rst_n; logic en; logic [3:0] count;`	The signals of the bundle. They are declared as `logic` variables. No directions are given here: an interface by itself only says which signals exist.
`endinterface`	Ends the declaration.
`module counter (cnt_if cif);`	The module has a single port named `cif` whose type is the interface `cnt_if`. This is called an **interface port**.

Line	Explanation
`cif.clk`, `cif.rst_n`, `cif.en`, `cif.count`	Signals inside the interface are accessed with `<port_name>.<signal_name>`.
`cif.count <= ...`	The design writes a variable that lives inside the interface. This is legal because `count` has exactly one writer, this `always_ff` block.

Complete example

Compile `counter_dut.sv` first, then this file.

tb_if_basic.sv

```
module tb_if_basic;

  logic clk = 0;
  always #5 clk = ~clk;

  cnt_if  cif (clk);     // interface instance: creates rst_n, en and count
  counter dut (cif);     // one connection carries all the signals

  initial begin
    cif.rst_n = 0;
    cif.en    = 0;
    @(negedge clk);
    cif.rst_n = 1;
    cif.en    = 1;

    repeat (3) @(negedge clk);
    $display("[%0t] count = %0d", $time, cif.count);

    cif.en = 0;
    repeat (2) @(negedge clk);
    $display("[%0t] count = %0d", $time, cif.count);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[40] count = 3
[60] count = 3
```

Line by line

Line	Explanation
`cnt_if cif (clk);`	Creates one instance of the interface, named `cif`, and connects the testbench clock to its `clk` port. From this line on, the signals `cif.rst_n`, `cif.en` and `cif.count` exist.
`counter dut (cif);`	Connects the whole bundle to the design in one go. The instance name in the testbench (`cif`) and the port name in the module (`cif`) happen to be the same here; they do not have to be.
`cif.rst_n = 0;`	The testbench drives an interface signal directly, using the instance name.
`$display(..., cif.count)`	The testbench reads the value the design wrote into the same interface.

Execution flow

Time	Event
0	`rst_n` goes from X to 0, which is a negative edge, so the counter is reset to 0. `en` = 0.

Time	Event
5	Rising clock edge. Reset is still active; `count` stays 0.
10	Falling edge. The testbench releases reset and sets `en` = 1.
15, 25, 35	Rising edges. `count` becomes 1, 2, 3.
40	Third falling edge after time 10. The testbench prints `count = 3` and sets `en` = 0.
45, 55	Rising edges with `en` = 0. `count` stays 3.
60	The testbench prints `count = 3` and ends the simulation.

The testbench changes its outputs on the **falling** edge and the design samples on the **rising** edge. This simple habit keeps the stimulus stable around the active edge and avoids a race. Part 4 replaces it with clocking blocks, which do the same job properly.

Ways to connect an interface instance to an interface port

```
counter dut (cif);          // by position
counter dut (.cif(cif));    // by name
counter dut (.cif);         // .name shorthand: instance name equals port name
counter dut (.*);           // implicit: connects ports to same-named instances and signals
```

> **Common mistake:** Forgetting the parentheses on an instance of an interface that has no ports. It is instantiated as `my_if bus ();` with empty parentheses. Without them the line is read as a variable declaration of an unknown type and fails to compile.

> **Common mistake:** Leaving an interface port unconnected. A module port of interface type must always be connected to an interface instance; unlike an ordinary port it cannot be left open.

Interview questions

Q1. How do you declare and instantiate an interface? Declare it with `interface name (ports); ... endinterface` outside any module. Instantiate it inside a module like a module instance: `name inst (port_connections);`.

Q2. How does a module use an interface? It declares a port whose type is the interface name, for example `module counter (cnt_if cif);`, and accesses the signals as `cif.en`, `cif.count`.

Q3. Are the signals copied when an interface is passed to two modules? No. There is one set of signals, inside the interface instance. Each module's interface port is a reference to that instance.

Q4. Where can an interface be declared? At the same level as a module: in the compilation unit, not inside a module, package or class. (The standard allows nested interface declarations, but they are almost never used.)

1.4 Interface Ports and Parameters

Interface ports

Some signals are not owned by the bus. The clock and often the reset are generated once and shared by many interfaces and modules. Such signals enter the interface through **ports**, declared like module ports:

```
interface bus_if (input logic clk, input logic rst_n);
  logic        valid;
  logic        ready;
  logic [31:0] data;
endinterface

module tb;
  logic clk = 0, rst_n;
  bus_if bus_a (clk, rst_n);            // both instances share the same clock and reset
  bus_if bus_b (.clk(clk), .rst_n(rst_n));
endmodule
```

Signal declared	Meaning
As an interface **port** (`input logic clk`)	Comes from outside. Every instance can be connected to the same source. Inside the interface it is used like any other signal: `bus_a.clk`
As an **internal signal** (`logic valid;`)	Belongs to this instance. Each instance has its own copy

Parameters

An interface can be parameterised exactly like a module, so one declaration serves buses of different widths.

```
interface mem_if #(parameter int AW = 4, parameter int DW = 8) (input logic clk);
  logic          we;
  logic [AW-1:0] addr;
  logic [DW-1:0] wdata;
  logic [DW-1:0] rdata;
endinterface

module tb;
  logic clk = 0;
  mem_if                    small_mem (clk);   // default: 4-bit address, 8-bit data
  mem_if #(.AW(16), .DW(32)) large_mem (clk);   // 16-bit address, 32-bit data
endmodule
```

> **Interview trap:** Two instances of the same interface with different parameter values are two **different types**. A virtual interface handle (Part 3) must be declared with the same parameter values as the instance it will point to: `virtual mem_if #(16, 32) vif;` . Assigning `large_mem` to a handle declared as plain `virtual mem_if` (default parameters) is a compile error. Exercise 3 in Part 6 shows a complete example.

Interview questions

Q1. Why is the clock usually a port of the interface instead of an internal signal? Because the clock is generated once, outside, and is shared by the DUT and by every interface. Making it a port lets all instances be connected to the same clock. Clocking blocks and assertions inside the interface need a clock to refer to, so it must be visible inside the interface.

Q2. Can an interface have parameters? Yes, with the same syntax as a module: `interface my_if #(parameter int W = 8) (input logic clk);`. Each distinct set of parameter values is a distinct type.

1.5 Connecting an Interface to a DUT

There are two styles of DUT, and you will meet both.

Style 1: the DUT has an interface port

The counter above is written this way. The connection is one line: `counter dut (cif);`.

Style 2: the DUT has ordinary ports

Most real RTL, and the ALU from Module 1, has ordinary ports. Many companies keep interfaces out of RTL port lists so that the netlist after synthesis has plain, predictable port names. The interface is then used on the **testbench side only**, and the instantiation connects each DUT port to the corresponding interface signal:

```
// First version of the ALU interface: signals only.
// Parts 2 and 4 add modports and clocking blocks to reach the final form.
interface alu_if (input logic clk);
  import alu_pkg::*;

  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;
endinterface

module tb_top;
  logic clk = 0;
  always #5 clk = ~clk;

  alu_if aif (clk);                   // the one and only bundle of ALU signals

  alu dut (                           // the DUT keeps its ordinary ports
    .clk       (clk),
    .rst_n     (aif.rst_n),
    .valid_in  (aif.valid_in),
    .a         (aif.a),
    .b         (aif.b),
    .op        (aif.op),
    .valid_out (aif.valid_out),
    .result    (aif.result)
  );

  // Everything else in the testbench receives "aif", never the individual signals.
endmodule
```

This still lists each signal, but only **once**, in the top module. The driver, the monitor and every other testbench component receive `aif` as a single item.

Style	Connection	Used when
DUT with an interface port	`counter dut (cif);`	The RTL team uses interfaces in the design
DUT with ordinary ports	`.a(aif.a), .b(aif.b), ...` once in `tb_top`	The RTL has plain ports (very common). The interface exists for the testbench

> **Verification application:** In a real project you receive the DUT as it is. If it has plain ports you write the interface yourself, with the same signal names as the DUT ports, and connect them in the top module exactly as above. Writing that interface is normally the first task when building a testbench for a new block.

Interview questions

Q1. The DUT has no interface in its port list. Can you still use an interface in the testbench? Yes. Instantiate the interface in the top module and connect each DUT port to the matching interface signal, for example `.a(aif.a)`. The rest of the testbench uses the interface.

Q2. Where is the interface instantiated? In the top-level testbench module, next to the DUT instance, because both the DUT and the testbench must be connected to it.

1.6 Clock and Reset Generation Inside Interfaces

In plain English

An interface may contain `initial` and `always` blocks, so it can generate a clock and apply a reset by itself. Whether it *should* depends on what the interface is for.

Syntax

```
interface clk_rst_if;

  logic clk = 0;
  logic rst_n;

  always #5 clk = ~clk;                       // clock generated inside the interface

  task automatic apply_reset(int unsigned cycles = 2);
    rst_n = 0;
    repeat (cycles) @(negedge clk);
    rst_n = 1;                                // released away from the rising edge
  endtask

endinterface
```

Complete example

Compile `counter_dut.sv` first, then this file.

tb_if_clkgen.sv

```
interface clk_rst_if;

  logic clk = 0;
  logic rst_n;

  always #5 clk = ~clk;

  task automatic apply_reset(int unsigned cycles = 2);
    rst_n = 0;
    repeat (cycles) @(negedge clk);
    rst_n = 1;
  endtask

endinterface

module tb_if_clkgen;

  clk_rst_if cr  ();              // generates clock and reset
  cnt_if     cif (cr.clk);        // the counter bus uses that clock
  counter    dut (cif);

  assign cif.rst_n = cr.rst_n;    // the counter bus uses that reset

  initial begin
    cif.en = 1;
    cr.apply_reset(2);
    $display("[%0t] reset released", $time);

    repeat (4) @(negedge cr.clk);
    $display("[%0t] count = %0d", $time, cif.count);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[20] reset released
[60] count = 4
```

Line by line

Line	Explanation
`interface clk_rst_if;`	An interface with no ports. It owns the clock and the reset.
`logic clk = 0; always #5 clk = ~clk;`	A free-running clock with a period of 10 time units, generated by the interface itself.
`task automatic apply_reset(...)`	A task inside the interface. It drives `rst_n` low, waits a number of falling edges, and drives it high.
`clk_rst_if cr ();`	Instance with empty parentheses because the interface has no ports.
`cnt_if cif (cr.clk);`	The clock of one interface is connected to the clock port of another.
`assign cif.rst_n = cr.rst_n;`	A continuous assignment drives the counter's reset from the clock/reset interface. `cif.rst_n` now has its one permitted driver, so the testbench must not also assign it procedurally.
`cr.apply_reset(2);`	A task of an interface instance is called as `<instance>.<task>`.

Execution flow

Time	Event
0	`apply_reset` drives `rst_n` = 0. The counter is reset. `en` = 1.
10, 20	Falling edges counted by the task. At 20 it releases the reset and returns.
25, 35, 45, 55	Rising edges with reset released and `en` = 1: `count` = 1, 2, 3, 4.
60	Fourth falling edge after time 20: prints `count = 4`.

The reset is released on a falling edge on purpose. Releasing it at exactly the rising edge would create a race between the testbench (changing `rst_n`) and the design (reading `rst_n`) in the same instant.

Should the clock be generated inside the interface?

Approach	Advantages	Disadvantages
Clock generated in `tb_top` and passed to interfaces as a port (the usual choice)	One clock source for the DUT and all interfaces. The bus interface stays free of testbench-only code, so it can also be used by RTL and be synthesised	A few more lines in `tb_top`
Clock generated inside the **bus** interface	Fewer lines	Every instance generates its own clock: two instances of the bus give two separate clocks. The interface can no longer be shared with synthesizable RTL. The clock frequency is fixed inside the interface
A dedicated clock-and-reset interface (as in the example)	Keeps clock control in one place; tasks such as `apply_reset`, `stop_clock`, `set_period` can be called from classes through a virtual interface	One more interface to write

> **Key point:** The bus interface (`alu_if`) receives the clock as an input port. The clock is generated once, in `tb_top` or in a dedicated clock interface. This is the arrangement used in the rest of the course and in almost every industrial testbench.

> **Interview trap:** "Can an interface contain an `always` block or an `initial` block?" Yes. Candidates often think an interface is only a list of wires. It can contain procedural blocks, continuous assignments, tasks, functions and assertions. What it cannot contain is a module instance.

Interview questions

Q1. Can you generate a clock inside an interface? Yes, for example `logic clk = 0; always #5 clk = ~clk;`. It is legal because an interface may contain procedural blocks.

Q2. Why is the clock normally not generated inside the bus interface? Because each instance of the interface would create its own independent clock, the DUT and the other interfaces need the same clock, and delay-based code makes the interface unusable in synthesizable RTL. The clock is generated once in the top module (or in a dedicated clock interface) and passed in as a port.

Q3. How would a test class control the reset? Through a virtual interface handle to an interface that owns the reset, calling a task such as `vif.apply_reset(5)` or assigning `vif.rst_n` directly.

1.7 Adding Values to Interface Signals

"Adding values" means driving the signals of the interface. There are several places a value can come from, and the single-driver rule from Module 1 decides which combinations are legal.

Who can drive an interface signal

Source	Example	Kind of assignment
Testbench code, by instance name	`cif.en = 1;` in an `initial` block of `tb_top`	Procedural
A module, through its interface port	`cif.count <= cif.count + 1;` in the DUT	Procedural
A module output port connected to the signal	`.result(aif.result)`	Port (continuous)
A continuous assignment	`assign cif.rst_n = cr.rst_n;`	Continuous
Code inside the interface	A task, an `initial` block or an `assign` in the interface	Either
A class, through a virtual interface	`vif.en = 1;` (Part 3)	Procedural
A clocking block output	`vif.drv_cb.a <= 5;` (Part 4)	Synchronous drive
A declaration initialiser	`logic en = 0;` inside the interface	Initial value at time 0

The rules

Interface signals declared as `logic` are **variables**, so the rules of Module 1, Section 2.4 apply:

Combination on one `logic` signal	Legal?
Any number of procedural assignments (testbench, tasks, classes)	Yes. The last value written wins
Exactly one continuous assignment or one module output port	Yes
A continuous assignment (or output port) **and** a procedural assignment	**No**: compile or elaboration error
Two continuous assignments or two output ports	**No**
Several drivers that must be resolved (bidirectional bus)	Only if the signal is declared as a net (`wire`); see Section 5.3

```
interface cnt_if (input logic clk);
  logic       rst_n;
  logic       en = 0;        // initial value: en is 0 from time 0 instead of X
  logic [3:0] count;
endinterface

module tb;
  logic clk = 0;
  cnt_if  cif (clk);
  counter dut (cif);          // dut writes cif.count procedurally

  initial begin
    cif.rst_n = 0;            // legal: procedural, only the testbench writes rst_n
    cif.en    = 1;            // legal
    // cif.count = 4'd7;      // BUG: count is the DUT's output. Here the tool reports it,
                              // because always_ff allows no second writer. With a plain
                              // "always" block in the DUT it would compile silently.
  end

  // assign cif.en = 1'b1;    // ILLEGAL: en would be driven continuously AND procedurally
endmodule
```

Common mistake: Leaving DUT inputs at X. Every interface signal that feeds the DUT starts as X. Give each one a value at time 0 (in the testbench's first statements, or with an initialiser in the interface), and apply reset before the first real stimulus.

Common mistake: Writing a DUT output from the testbench. Whether the tool notices depends on how the DUT drives the signal: an output port or an `always_ff` block gives an error, but two plain procedural writers are accepted silently and simply overwrite each other. Modports (Part 2) make the intended direction explicit, so the mistake is reported where it is made.

Interview questions

Q1. How does a testbench drive a signal that is inside an interface? By hierarchical name from a module (`cif.en = 1;`), through an interface port, from a task inside the interface, or from a class through a virtual interface (`vif.en = 1;` or, with a clocking block, `vif.drv_cb.en <= 1;`).

Q2. An interface signal is driven by `assign` inside the interface. Can the testbench also assign it in an `initial` block? No, if the signal is a `logic` variable. A variable may be driven either by one continuous assignment or by procedural assignments, not both.

Q3. What is the initial value of the signals in an interface? X for `logic` signals and Z for undriven nets, unless the declaration has an initialiser.

1.8 Tasks, Functions and Assertions Inside Interfaces

This section is for awareness. The class-based testbench in Module 9 keeps the driving code in the driver class, but you must be able to recognise and explain code written in the style below.

Tasks and functions: a bus functional model

A task inside an interface can perform a complete bus operation. An interface that contains such tasks is called a **bus functional model (BFM)**: the user calls `write(addr, data)` and the interface wiggles the signals.

```
interface mem_if (input logic clk);
  logic       we;
  logic [3:0] addr;
  logic [7:0] wdata;
  logic [7:0] rdata;

  // One complete write cycle
  task automatic write(input logic [3:0] wr_addr, input logic [7:0] wr_data);
    @(negedge clk);
    we    = 1;
    addr  = wr_addr;
    wdata = wr_data;
    @(negedge clk);
    we    = 0;
  endtask

  // A function that answers a question about the current state of the bus
  function automatic bit is_idle();
    return (we === 1'b0);
  endfunction
endinterface
```

A testbench calls `mif.write(4'h3, 8'hAB);`. A class calls the same task through its virtual interface: `vif.write(4'h3, 8'hAB);`.

Assertions

An assertion placed in the interface checks the protocol wherever the interface is used: in block-level simulation, at chip level, and in formal verification. Assertions are the subject of Module 10; this is only the idea.

```
interface alu_if (input logic clk);
  // ... signals ...

  // After reset, valid_in must never be X or Z
  property p_valid_known;
    @(posedge clk) disable iff (!rst_n) !$isunknown(valid_in);
  endproperty
  a_valid_known: assert property (p_valid_known)
    else $error("valid_in is X or Z");

  // A result must be flagged exactly one cycle after a valid request
  a_latency: assert property (@(posedge clk) disable iff (!rst_n)
                              valid_in |=> valid_out);
endinterface
```

Tasks in the interface versus tasks in the driver class

Property	Task inside the interface (BFM style)	Task inside the driver class
Where the pin-level protocol is written	In the interface	In the class
Can be overridden for a special test (inheritance)	No. An interface cannot be extended	Yes. A derived driver can override the task
Works in emulation (hardware acceleration)	Yes, the interface can be synthesised into the emulator	No, classes are not synthesizable
Usual in UVM testbenches	Only when emulation is a requirement	Yes, the standard approach

Interview questions

Q1. Can an interface contain tasks and functions? Give a use. Yes. A task can implement one bus operation, such as a write cycle, so that all users call the task instead of driving signals individually. Such an interface is called a bus functional model.

Q2. Why put assertions in an interface? The protocol rules belong to the bus, not to one particular testbench. An assertion in the interface is active everywhere the interface is instantiated, at block level and at chip level, with no extra work.

Q3. Can an interface be extended like a class? No. Interfaces have no inheritance. This is one reason the driving code is normally written in a class, where it can be overridden.

Part 2: Modports and Interface Arrays

2.1 What Is a Modport and Why Is It Required?

In plain English

The interface in Part 1 says which signals exist. It does not say who is allowed to drive them. The counter drives `count`; the testbench drives `rst_n` and `en`. Nothing in `cnt_if` records that, so nothing stops the testbench from writing `count` or the design from writing `en` by mistake.

A **modport** (short for "module port") is a named view of the interface for one kind of user. It lists the signals that user may access and gives each one a direction **as seen by that user**. An interface normally has one modport per role: one for the DUT, one for the testbench or driver, one for the monitor.

Technical definition

A `modport` is a declaration inside an interface that defines, for a named role, a subset of the interface's signals together with their port directions (`input`, `output`, `inout`, `ref`). When a module port, an instance connection or a virtual interface selects that modport, access through it is restricted to the listed signals in the listed directions.

Why it is required

Without modports	With modports
Every user has read and write access to every signal	Each role sees only its own signals, with a fixed direction
A write to the wrong signal is found late, in simulation, as contention or wrong data	It is a compile or elaboration error at the offending line
Direction of each signal is known only from comments	Direction is part of the code and is checked
A synthesis tool cannot tell inputs from outputs on an interface port	The modport gives the synthesis tool the port directions
A monitor could drive the bus by accident	The monitor's modport has only inputs, so a monitor cannot drive anything

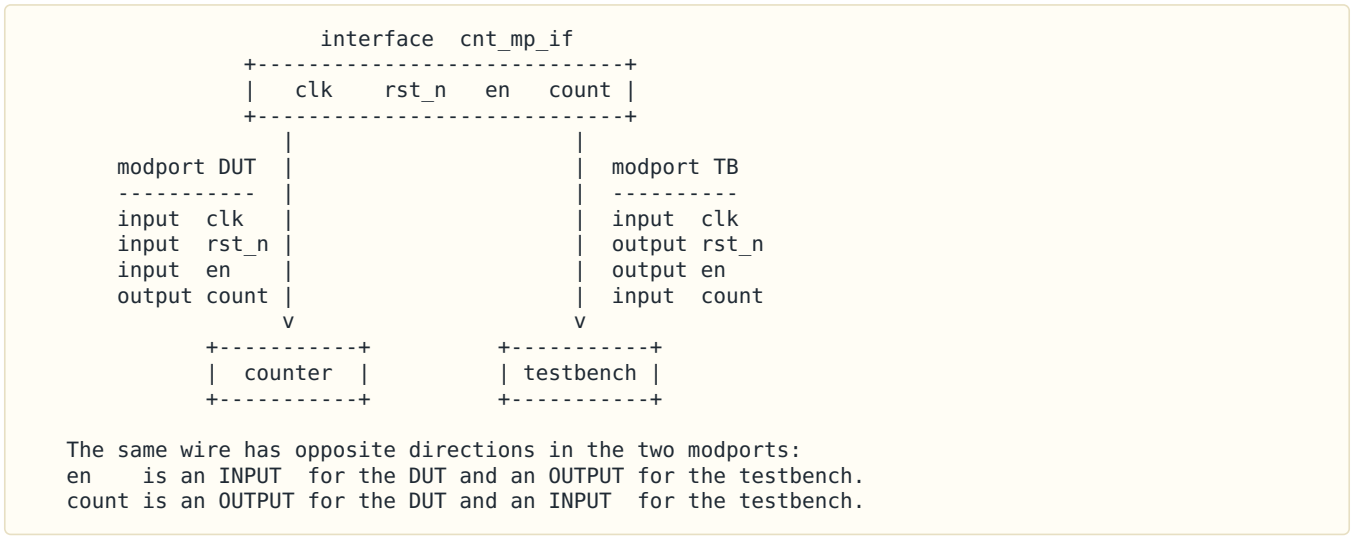
Syntax

```
interface <name> (...);
  <signals>

  modport <modport_name> (
    input  <signal>, <signal>,
    output <signal>,
    inout  <signal>,
    ref    <signal>
  );
endinterface
```

A direction keyword applies to every name after it until the next direction keyword. Several modports may list the same signal with different directions; that is the whole point.

A first picture



Interview questions

Q1. What is a modport? A named list, inside an interface, of the signals that one kind of module may access and the direction of each signal from that module's point of view.

Q2. Why are modports needed? An interface alone gives every connected block full access to all signals. Modports define direction, so that a block cannot drive a signal it should only read, different roles (DUT, driver, monitor) get different views of the same signals, and synthesis tools know which interface signals are inputs and outputs of a module.

Q3. Does a modport create new signals? No. It creates nothing. It is a set of access rules for signals that already exist in the interface.

2.2 Input, Output and Inout Directions in a Modport

The four directions

Directions are always written **from the point of view of the module that uses the modport**.

Direction	The user may read	The user may write	Typical use
`input`	Yes	**No**: an assignment is an error	Signals this role only observes
`output`	Yes	Yes	Signals this role drives
`inout`	Yes	Yes	Bidirectional nets (`wire`) such as a shared data bus
`ref`	Yes	Yes	A variable shared by reference: every user sees every change immediately

Signals that are **not listed** in a modport cannot be accessed through it at all. A modport therefore also hides signals.

What a modport enforces, and what it does not

This is asked often, because many people assume modports do more than they do.

A modport does	A modport does not
Make writing an `input` signal through that modport an error	Stop code that bypasses the modport. A hierarchical reference such as `tb_top.cif.count = 0;` is still accepted
Make any access to an unlisted signal through that modport an error	Check that an `output` is actually driven by somebody
Tell synthesis the direction of each signal of an interface port	Add buffers, drivers or any hardware
Restrict a virtual interface declared with that modport	Define any timing: when to sample or drive is the job of a clocking block (Part 4)
Document the role of each signal	Prevent two users from both selecting a modport with the same `output`. That is caught, if at all, by the ordinary multiple-driver rules

> **Key point:** A modport is checked only when the access goes **through** it: through a module port declared with the modport, an instance connected with the modport, or a virtual interface declared with the modport. If nobody selects a modport, it has no effect.

What happens when no modport is selected

If a module port is declared with just the interface name (`cnt_if cif`) and the instance is connected without a modport, the module gets full access: nets behave as `inout` and variables behave as `ref`. This is how the examples in Part 1 worked.

Interview questions

Q1. From whose point of view are modport directions written? From the point of view of the module (or class) that uses the modport. A signal the DUT drives is an `output` in the DUT's modport and an `input` in the testbench's modport.

Q2. What happens if a module writes a signal that its modport declares as `input`? It is an error at compile or elaboration time.

Q3. Can a testbench still write a signal that its modport does not allow? Yes, if it bypasses the modport by using the full hierarchical name of the interface instance. Modports restrict access only through the port or virtual interface that selected them.

Q4. A signal is not listed in the modport. Can the module read it? No. Only listed signals are visible through a modport.

2.3 Different Modports for the DUT and the Testbench

The counter interface with modports

This file is a second version of the counter and its interface, with modports added. It is separate from `counter_dut.sv` so that the Part 1 examples stay unchanged.

counter_mp.sv

```
interface cnt_mp_if (input logic clk);
  logic       rst_n;
  logic       en;
  logic [3:0] count;

  // View for the design: it reads the controls and drives the count
  modport DUT (input  clk, rst_n, en,
               output count);

  // View for the testbench: it drives the controls and reads the count
  modport TB  (input  clk, count,
               output rst_n, en);
endinterface

module counter_mp (cnt_mp_if.DUT cif);       // this module uses the DUT view

  always_ff @(posedge cif.clk or negedge cif.rst_n) begin
    if (!cif.rst_n)  cif.count <= '0;
    else if (cif.en) cif.count <= cif.count + 1'b1;
  end

  // assign cif.en = 1'b1;        // ILLEGAL: en is an input in modport DUT

endmodule
```

Complete example

Compile `counter_mp.sv` first, then this file.

tb_modport.sv

```
// A stimulus module that uses the testbench view of the interface
module cnt_stim (cnt_mp_if.TB cif);

  initial begin
    cif.rst_n = 0;
    cif.en    = 0;
    @(negedge cif.clk);
    cif.rst_n = 1;
    cif.en    = 1;

    repeat (5) @(negedge cif.clk);
    $display("[%0t] count = %0d", $time, cif.count);

    // cif.count = 4'd0;          // ILLEGAL: count is an input in modport TB
    $finish;
  end

endmodule

module tb_modport;

  logic clk = 0;
  always #5 clk = ~clk;

  cnt_mp_if  cif  (clk);
  counter_mp dut  (cif);          // modport DUT was chosen in the module header
  cnt_stim   stim (cif.TB);       // the modport may also be named at the instance

endmodule
```

EXPECTED OUTPUT

```
[60] count = 5
```

Line by line

Line	Explanation
`modport DUT (input clk, rst_n, en, output count);`	The design's view. `clk` is a port of the interface; ports may be listed in a modport like any other signal.
`modport TB (input clk, count, output rst_n, en);`	The testbench's view: exactly the opposite directions for `rst_n`, `en` and `count`.
`module counter_mp (cnt_mp_if.DUT cif);`	The port type is `<interface>.<modport>`. Inside this module, `cif` obeys modport `DUT`.
`// assign cif.en = 1'b1;`	Remove the comment marker and the compiler rejects the line, because `en` is an input for this module.
`module cnt_stim (cnt_mp_if.TB cif);`	A second module, with the opposite view. It may write `rst_n` and `en` but not `count`.
`cnt_stim stim (cif.TB);`	The modport is named again in the connection. This is allowed when it is the same modport as in the module header; it is only extra documentation here.

Execution flow

Time	Event
0	`rst_n` = 0, `en` = 0: the counter is reset.
10	Falling edge: reset released, `en` = 1.
15, 25, 35, 45, 55	Five rising edges: `count` = 1, 2, 3, 4, 5.
60	Fifth falling edge after time 10: prints `count = 5`.

Typical modport sets

Interface for	Modports	Notes
A block and its testbench	`DUT`, `TB`	The simplest arrangement, as above
A class-based testbench	`DUT`, `DRV`, `MON`	The driver's view drives the DUT inputs; the monitor's view has only inputs. This is the arrangement of `alu_if`
A bus between two design blocks	`master` / `slave`, or `manager` / `subordinate`, `initiator` / `target`	Request signals are outputs of one side and inputs of the other; response signals the reverse

```
// A valid/ready handshake bus between two design blocks
interface stream_if (input logic clk, input logic rst_n);
  logic        valid;
  logic        ready;
  logic [31:0] data;

  modport source (input clk, rst_n, ready, output valid, data);   // sends data
  modport sink   (input clk, rst_n, valid, data, output ready);   // receives data
  modport mon    (input clk, rst_n, valid, ready, data);          // only observes
endinterface
```

The ALU interface with modports (second version)

```
interface alu_if (input logic clk);
  import alu_pkg::*;

  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  modport DUT (input clk, rst_n, valid_in, a, b, op, output valid_out, result);
  // The DRV and MON modports refer to clocking blocks and are added in Part 4.
endinterface
```

Common mistake: Writing the directions from the wrong side. In the testbench modport, a signal that the testbench drives **into** the DUT is an `output`, even though it is an input of the DUT. Read every modport as "I am this role; what do I drive, what do I read?".

Verification application: The monitor must be passive: it may never change a DUT signal. Giving the monitor a modport that contains only inputs makes that a compile-time guarantee instead of a coding guideline.

Interview questions

Q1. Why do the DUT and the testbench need different modports? Because their directions are opposite. The DUT's inputs are driven by the testbench and the DUT's outputs are read by it. One modport cannot describe both views.

Q2. In the testbench modport, is a DUT input signal an `input` or an `output`? An `output`, because the testbench drives it.

Q3. Why should a monitor have its own modport? So that all signals are inputs for the monitor. Any accidental attempt by the monitor to drive the bus then fails to compile.

2.4 Selecting the Modport: Module Header or Instance

There are two places where a modport can be chosen.

Method 1: in the module header (preferred)

```
module counter_mp (cnt_mp_if.DUT cif);     // the module itself declares its role
  ...
endmodule

counter_mp dut (cif);                      // plain connection
```

Method 2: at the instantiation

```
module counter_any (cnt_mp_if cif);        // no modport in the header
  ...
endmodule

counter_any dut (cif.DUT);                 // the role is chosen where the module is used
```

Comparison

Property	In the module header	At the instance
Syntax	`module m (my_if.MP port);`	`m inst (my_if_inst.MP);`
Who decides the role	The author of the module	The person who instantiates it
The module can be used with only one role	Yes: the restriction travels with the module	No: each instance may choose a different modport
Protection if the instantiator forgets the modport	Still protected	None: without `.DUT` the module has full access
Both specified	Allowed only if they name the **same** modport; otherwise an error	
Neither specified	Full access to every signal	
Usual practice	RTL modules and testbench components	Rare

> **Interview trap:** "If the module header says `cnt_mp_if.DUT` and the instance connects `cif.TB`, which one wins?" Neither. It is an error. When a modport is given in both places it must be the same one.

Selecting a modport in a virtual interface

A class has no ports, so the third place to select a modport is the declaration of a virtual interface handle (Part 3):

```
virtual cnt_mp_if.TB vif;        // a class property restricted to the TB view
```

Interview questions

Q1. In how many ways can a modport be selected? Three: in the port declaration of the module (`module m (my_if.MP p);`), in the port connection of the instance (`m i (bus.MP);`), and in the declaration of a virtual interface (`virtual my_if.MP vif;`).

Q2. Which way is preferred for a module, and why? In the module header. The module then always has the correct restricted view, whoever instantiates it, and a reader of the module sees its role immediately.

2.5 Accessing Interface Signals

This section collects every way of reaching a signal that lives inside an interface. Each has been or will be shown in an example.

Where the code is	How it names the signal	Restricted by a modport?	Section
In the module that instantiates the interface	`<instance>.<signal>`: `cif.en`	No	1.3
Anywhere, by full hierarchical path	`tb_top.cif.en`	No	2.2
In a module with an interface port	`<port>.<signal>`: `cif.en`	Yes, if the port or the connection names a modport	2.3, 2.4
Inside the interface (its own tasks, assertions, `always` blocks)	The plain name: `en`	No	1.8
In a class	`<virtual interface handle>.<signal>`: `vif.en`	Yes, if the handle is declared with a modport	Part 3
Through a clocking block	`<handle>.<clocking block>.<signal>`: `vif.drv_cb.a`	Yes, and also by the clocking block's own directions	Part 4
An element of an interface array	`<instance>[<constant index>].<signal>`: `cif[1].en`	No	2.7
A task or function of the interface	`<instance or handle>.<task>(...)`: `vif.write(...)`	Yes: a modport must `import` the task to make it callable	2.6

> **Common mistake:** Using the interface **type** name instead of an **instance** name: `cnt_if.en = 1;`. `cnt_if` is a declaration. Signals exist only in instances (`cif.en`).

> **Common mistake:** Reading or writing an interface signal from a class by hierarchical path (`tb_top.cif.en = 1;`). It may compile if the class is declared in the same file, but it fails when the class is in a package, and it ties the class to one particular testbench. A class must use a virtual interface.

Interview questions

Q1. How do you access a signal inside an interface from a module? With the dot operator on the interface instance or interface port: `<instance_or_port>.<signal>`.

Q2. How do you access it from a class? Through a virtual interface handle: `vif.<signal>`, or `vif.<clocking_block>.<signal>` when a clocking block is used.

2.6 Tasks and Clocking Blocks in Modports

This section is for awareness.

Clocking blocks in a modport

A modport may list a clocking block instead of individual signals. The user of that modport can then reach those signals only through the clocking block, which forces synchronous access. This is how the final `alu_if` gives the driver and the monitor their views:

```
modport DRV (clocking drv_cb, input clk, rst_n);
modport MON (clocking mon_cb, input clk, rst_n);
```

Part 4 explains clocking blocks and comes back to these two lines.

Importing tasks and functions

A task or function declared in the interface is not visible through a modport unless the modport imports it:

```
interface mem_if (input logic clk);
  logic       we;
  logic [3:0] addr;
  logic [7:0] wdata, rdata;

  task automatic write(input logic [3:0] wr_addr, input logic [7:0] wr_data);
    @(negedge clk);  we = 1;  addr = wr_addr;  wdata = wr_data;
    @(negedge clk);  we = 0;
  endtask

  modport TB  (import write, input clk, rdata);          // may call write()
  modport DUT (input clk, we, addr, wdata, output rdata); // cannot call write()
endinterface
```

The opposite keyword, `export`, lets a module that is connected to the interface provide the body of a task, so that other users of the interface can call it. It is rarely used.

Interview questions

Q1. Can a modport contain something other than signals? Yes. It can list a clocking block (`clocking cb_name`), and it can import tasks and functions of the interface (`import task_name`) so that they can be called through the modport.

Q2. A task is declared in an interface. A module connected through a modport cannot call it. Why? The modport does not import the task. Only what a modport lists is accessible through it; add `import <task_name>` to the modport.

2.7 Interface Arrays

In plain English

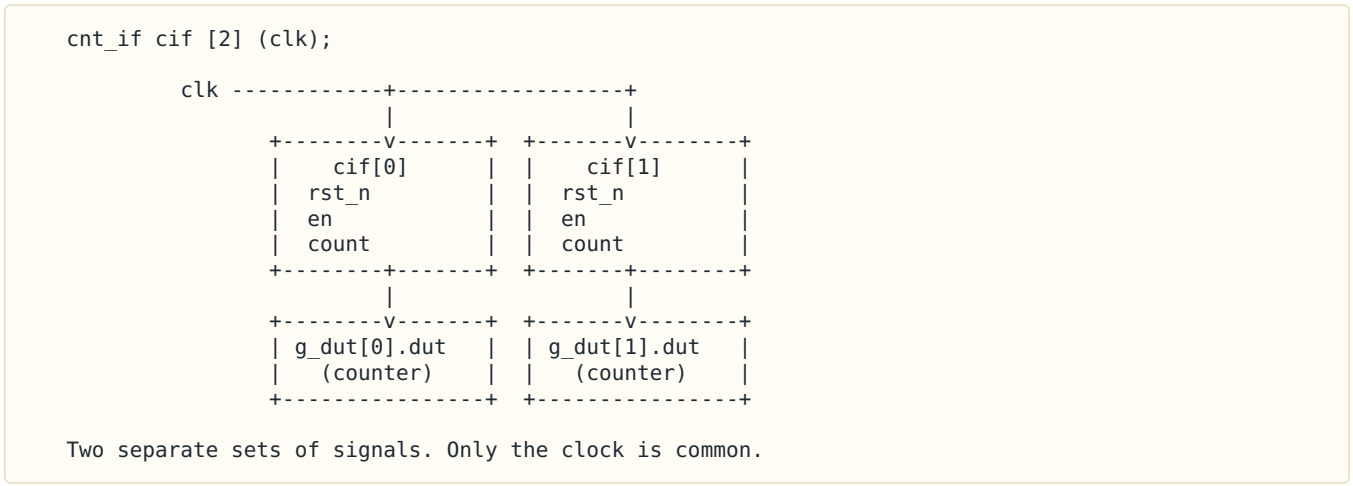
A design often has several identical ports: a switch with four input channels, a system with eight identical peripherals. Each needs its own copy of the bus. Instead of declaring `bus0`, `bus1`, `bus2`, `bus3`, you declare an **array of interface instances**.

Syntax

```
cnt_if cif [4] (clk);          // four instances: cif[0], cif[1], cif[2], cif[3]
cnt_if cif [0:3] (clk);        // the same, with an explicit range
```

- Each element is a complete, independent instance with its own signals.
- A single port connection such as `(clk)` is connected to **every** element, so all four instances share the clock.
- An element is selected with a **constant** index: `cif[2].en`.

How it works internally



An interface array is an array of **instances**, created at elaboration like an array of module instances. It is not an unpacked array variable: it has no array methods, it cannot be resized, and it cannot be indexed with a variable.

Complete example

Compile `counter_dut.sv` first, then this file.

tb_if_array.sv

```
module tb_if_array;

  logic clk = 0;
  always #5 clk = ~clk;

  cnt_if cif [2] (clk);                  // two interface instances

  for (genvar i = 0; i < 2; i++) begin : g_dut
    counter dut (cif[i]);                // one counter per interface instance
  end

  initial begin
    cif[0].rst_n = 0;   cif[1].rst_n = 0;
    cif[0].en    = 1;   cif[1].en    = 0;
    @(negedge clk);
    cif[0].rst_n = 1;   cif[1].rst_n = 1;

    repeat (2) @(negedge clk);
    cif[1].en = 1;                       // the second counter starts later

    repeat (2) @(negedge clk);
    $display("[%0t] count0 = %0d  count1 = %0d", $time, cif[0].count, cif[1].count);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[50] count0 = 4  count1 = 2
```

Line by line

Line	Explanation
`cnt_if cif [2] (clk);`	Creates `cif[0]` and `cif[1]`. Both receive the same `clk`.

Line	Explanation
`for (genvar i ...) begin : g_dut`	A generate loop. It is evaluated at elaboration and creates two counter instances, `g_dut[0].dut` and `g_dut[1].dut`.
`counter dut (cif[i]);`	Inside a generate loop the index `i` is a constant for each iteration, so `cif[i]` is legal here.
`cif[0].en = 1; cif[1].en = 0;`	Each instance has its own `en`.

Execution flow

Time	Counter 0 (`en` = 1 from the start)	Counter 1
0 to 10	In reset, 0	In reset, 0
15, 25	1, 2	`en` = 0: stays 0
30		`en` set to 1
35, 45	3, 4	1, 2
50	Printed: 4	Printed: 2

The constant-index rule

```
initial begin
  for (int i = 0; i < 2; i++)
    cif[i].en = 1;      // ILLEGAL: i is a variable; an instance needs a constant index
end
```

`cif[i]` is part of a hierarchical name, and the hierarchy is fixed at elaboration. Selecting an instance with a run-time variable is therefore not allowed. The working alternatives are:

Need	Solution
Connect each element to a module	A `generate` loop with a `genvar`, as in the example
Do the same thing to every element from procedural code	An **array of virtual interfaces**, which is an ordinary array of handles and can be indexed with a variable (Section 3.7)
A few elements	Write the constant indices out: `cif[0].en = 1; cif[1].en = 1;`

Interview trap: "Can you loop over an array of interfaces with a `for` loop inside an `initial` block?" Not directly: the index of an instance array must be a constant expression. The standard answer is to copy the instances into an array of virtual interfaces and loop over that.

Verification application: A testbench for a 4-port switch has `port_if pif [4] (clk);`, connects `pif[i]` to DUT port `i` in a generate loop, and creates four agents. Each agent receives one element of an array of virtual interfaces, so the same driver and monitor classes serve all four ports.

Interview questions

Q1. How do you declare an array of interfaces? `my_if bus [N] (clk);` creates N independent instances, `bus[0]` to `bus[N-1]`. A single port connection is shared by all elements.

Q2. Can the index of an interface array be a variable? No. It must be a constant, a parameter or a `genvar`, because the element is an instance in the static hierarchy. To use a variable index, use an array of virtual interfaces.

Q3. Do the elements of an interface array share signals? No. Each element has its own copy of all internal signals. Only signals connected through the interface's ports (such as the clock) are common.

2.8 Generic Interface Ports

For awareness: a module port can be declared with the keyword `interface` instead of a specific interface name. The module then accepts **any** interface, and the actual type is fixed when the module is instantiated.

```
module generic_probe (interface bus);            // any interface may be connected
  always @(posedge bus.clk)
    if (bus.valid) $display("[%0t] valid seen", $time);
endmodule

generic_probe p1 (stream_bus);                   // works if stream_bus has clk and valid
```

The module compiles for any connected interface that has the signals it uses (`clk` and `valid` here); otherwise elaboration fails. A modport can be chosen at the instance (`generic_probe p1 (stream_bus.mon);`) or in the header (`interface.mon bus`). Generic interface ports are rare in practice because the errors appear late and are harder to read.

Interview questions

Q1. What is a generic interface port? A module port declared as `interface <name>` without naming an interface type. Any interface instance may be connected to it, provided it contains the signals the module refers to.

2.9 Summary of Part 2

Concept	One-line summary
Modport	A named view of an interface: which signals, in which direction, for one role
Direction	Written from the user's side: what the testbench drives is an `output` in the testbench modport
Selection	In the module header (preferred), at the instance, or in a virtual interface declaration
Enforcement	Only for access through the modport. Hierarchical references bypass it
Not listed	Not accessible through that modport
Interface array	Several independent instances; constant index only
Modport with a clocking block	Forces the user to go through the clocking block (Part 4)

Part 3: Virtual Interfaces

3.1 The Problem: A Class Cannot Reach a Physical Interface

In plain English

Everything so far used **modules** to drive the interface. But the testbench of this course, and every modern testbench, is built from **classes**: the driver and the monitor are objects created with `new()` while the simulation runs (Module 4). The interface, on the other hand, is part of the design hierarchy and is created before the simulation starts.

So there are two worlds:

	Static world	Dynamic world
Contains	Modules, interfaces, the DUT, wires	Class objects: transactions, generator, driver, monitor, scoreboard
Created	At elaboration, before time 0	At run time, by calling `new()`
Lifetime	The whole simulation	From `new()` until no handle refers to the object
Number of copies	Fixed by the source code	Decided while the simulation runs
Connected by	Ports	Handles

The driver class lives in the dynamic world. The DUT pins live in the static world. The driver must wiggle those pins. How does it reach them?

Three things that do not work

Attempt 1: instantiate the interface inside the class.

```
class cnt_driver;
  cnt_if cif (clk);          // ILLEGAL: an interface cannot be instantiated in a class
endclass
```

An interface instance is a piece of hierarchy that must exist at elaboration so that it can be wired to the DUT. A class object does not exist at elaboration, and there could be zero or a thousand objects of the class. The language forbids this.

Attempt 2: give the class ports.

A module receives an interface through a port. A class has no ports. Its only inputs are the arguments of its methods and constructor, and an interface instance is not a value that can be passed as an ordinary argument.

Attempt 3: use the hierarchical name from inside the class.

```
class cnt_driver;
  task run();
    tb_vif_basic.cif.en = 1;     // hard-coded path to one particular testbench
  endtask
endclass
```

Problem with the hierarchical name	Explanation
Not allowed in a package	Classes are normally declared in a package, and code in a package may not contain hierarchical references to design instances
Not reusable	The class works only in a testbench whose top module and instance have exactly these names. At chip level the path is different
One instance only	Two drivers for two buses would need two different classes, identical except for the path
The path is fixed when the code is compiled	A test cannot choose at run time which interface a driver should use

What is needed

The class needs a **variable** that can be made to refer to an interface instance: set from outside, when the object is built, to whichever instance this object should work with. That variable is a virtual interface.

Interview questions

Q1. Why can a class not instantiate an interface? An interface instance is a static design element: it is created at elaboration and must be physically connected to the DUT. A class object is dynamic: it is created at run time by `new()`, and any number of objects may exist. A static hierarchy element cannot be created inside a dynamic object, so the language does not allow it.

Q2. Why not access the interface from the class with a hierarchical path? Code inside a package cannot use hierarchical references, and even where it compiles, the class becomes tied to one testbench hierarchy and one interface instance. It cannot be reused at a different level or for a second instance of the bus.

3.2 What Is a Virtual Interface?

In plain English

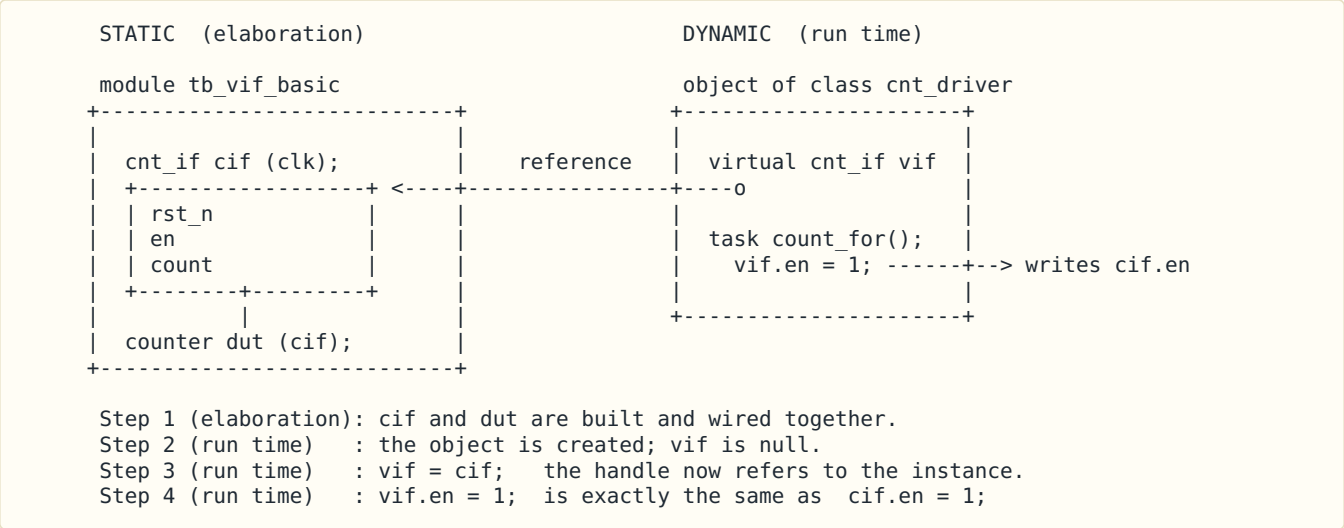
A virtual interface is a **handle to an interface instance**: a variable that stores "which interface am I talking to". It is not an interface and it contains no signals. Once it has been pointed at a real interface instance, writing `vif.en = 1` changes the `en` signal of that instance.

The interface instance in the top module is often called the **physical interface** (or actual interface), and the handle is the **virtual interface**.

Technical definition

A virtual interface is a variable, declared with the keyword `virtual` followed by an interface type, that holds a reference to an instance of that interface type. Its default value is `null`. It can be assigned an interface instance or another virtual interface of the same type, compared with `==` and `!=`, passed as an argument to tasks and functions, and stored as a class property. Through it, all signals, clocking blocks, tasks and functions of the referenced instance can be accessed.

How it works internally



- The handle stores only a reference. No signal is copied, and nothing is delayed.
- Several handles may refer to the same instance. A change made through one is seen through all, because there is only one set of signals.
- One handle can be re-pointed to a different instance of the same interface type while the simulation runs.

Why the word "virtual"?

The word is used in three unrelated places in SystemVerilog. Interviewers like to check that you do not confuse them.

Construct	Meaning of `virtual`	Module
`virtual interface` (`virtual cnt_if vif;`)	A handle that refers to an interface instance	8
`virtual class`	An abstract class: it cannot be constructed, only extended	4
`virtual function` / `virtual task`	A method that can be overridden, with the call resolved by the type of the object at run time	4

Physical interface versus virtual interface

Property	Interface instance (physical)	Virtual interface
What it is	A design element containing the signals	A variable holding a reference
Declared as	`cnt_if cif (clk);`	`virtual cnt_if vif;`
Declared where	In a module or in another interface	In a class, or as a variable in a module, program, task or function
Created	At elaboration	When its variable or object is created
Initial state	Exists, signals are X	`null`: refers to nothing
Contains signals	Yes	No

Property	Interface instance (physical)	Virtual interface
Can be changed at run time to refer to something else	No	Yes
Can be passed as a task/function argument	No	Yes
Can be a class property	No	Yes
Can be stored in dynamic arrays, queues, associative arrays	No	Yes
Can be a module port	Yes (interface port)	No
Synthesizable	Yes	No

Interview questions

Q1. What is a virtual interface? A variable that holds a reference (handle) to an instance of an interface. It lets dynamic, class-based code access the signals of a static interface instance.

Q2. Why is it needed? Classes cannot have ports and cannot instantiate interfaces, and hierarchical paths make them non-reusable. A virtual interface is a property of the class that is assigned from outside, so the same driver or monitor class works with any instance of the interface, at any level of hierarchy.

Q3. Does a virtual interface contain signals? No. It is only a reference. The signals exist once, in the interface instance.

Q4. What is the difference between an interface and a virtual interface? An interface instance is a static design element that holds the signals. A virtual interface is a dynamic handle that points to such an instance; it is `null` until assigned and can be re-assigned at run time.

Q5. What are the three uses of the keyword `virtual`? Virtual interface (a handle to an interface instance), virtual class (an abstract class), and virtual method (a method that is overridden polymorphically).

3.3 Virtual Interface Declaration

Syntax

```
virtual <interface_type> <handle_name>;                    // the usual form
virtual interface <interface_type> <handle_name>;         // keyword interface is optional
virtual <interface_type>.<modport> <handle_name>;          // restricted to one modport
virtual <interface_type> #(<parameter values>) <handle_name>; // parameterised interface
```

Examples

```
class cnt_driver;
  virtual cnt_if vif;                    // class property (the most common place)
endclass

class alu_driver;
  virtual alu_if.DRV vif;                // only what modport DRV allows is accessible
endclass

class mem_driver;
  virtual mem_if #(.AW(16), .DW(32)) vif; // parameters must match the instance exactly
endclass

module tb;
  virtual cnt_if vif_a;                  // a variable in a module
  virtual cnt_if vifs [4];               // an array of handles
  virtual cnt_if vif_q [$];              // a queue of handles

  task automatic pulse_enable(virtual cnt_if v);   // a task argument
    v.en = 1;
    @(negedge v.clk);
    v.en = 0;
  endtask
endmodule
```

Where a virtual interface may and may not be used

Allowed	Not allowed
Property of a class	Port of a module or interface
Variable in a module, program, task, function or procedural block	Member of a union
Argument and return value of tasks and functions	Left-hand side of a continuous assignment
Element of fixed, dynamic and associative arrays and queues	Any synthesizable code
Member of a struct or of another class (for example a configuration object)	

The interface type must already be known to the compiler, so the file that declares the interface is compiled before the classes that refer to it.

> **Key point:** By convention the handle is named `vif` in every class of a testbench. This course always uses that name.

Interview questions

Q1. How do you declare a virtual interface? `virtual <interface_name> <handle>;`, for example `virtual alu_if vif;`. The keyword `interface` may be written after `virtual` but is optional.

Q2. Can a virtual interface be declared with a modport? Yes: `virtual alu_if.DRV vif;`. Access through the handle is then limited to what that modport allows.

Q3. What is the default value of a virtual interface? `null`.

3.4 Assigning a Physical Interface to a Virtual Interface

The legal assignments

```
virtual cnt_if vif, vif2;

vif  = cif;          // an interface instance
vif  = cif_arr[1];   // an element of an interface array (constant index)
vif2 = vif;          // another virtual interface of the same type
vif  = null;         // refers to nothing again

if (vif == null) ...     // comparison with null
if (vif == vif2) ...     // do both handles refer to the same instance?
```

Right-hand side	Result
An interface instance of the same type (and same parameter values)	The handle refers to that instance
Another virtual interface of the same type	Both handles refer to the same instance
`null`	The handle refers to nothing
An instance of a different interface type, or with different parameters	Compile error

Where the assignment happens

The assignment must be written where the interface instance is **visible by name**: in the module that instantiates it, which is the top-level testbench module. This is the single point at which the static world is handed to the dynamic world.

```
tb_top (module)                      classes
-------------------------------------------------------------------
alu_if aif (clk);
alu    dut (... aif.a ...);

initial begin
  test = new(aif);        ------->   alu_test::new(virtual alu_if vif)
                                        env = new(vif);      ------>  alu_env::new(vif)
                                                                         drv = new(vif);
                                                                         mon = new(vif);

  test.run();
end
```

The handle is passed down from the top module through the constructors until it reaches the components that need it. Section 3.6 lists the alternatives.

Modports in the assignment

Handle declared as	May be assigned
`virtual alu_if vif;`	An instance (`aif`) or a handle without a modport
`virtual alu_if.DRV vif;`	An instance (`aif`), a handle without a modport, the same modport of an instance (`aif.DRV`), or a handle with the same modport

A handle that is restricted to a modport cannot be assigned to an unrestricted handle, and cannot be assigned to a handle with a different modport: that would remove or change the restriction.

> **Interview trap:** "Where do you connect the virtual interface to the physical interface?" In the top-level module, because that is the only place where the interface instance exists by name. The class never mentions the instance; it only receives a handle.

Interview questions

Q1. How is a physical interface assigned to a virtual interface? With an ordinary assignment, `vif = aif;`, made in the module where the instance `aif` is declared, or by passing `aif` as the actual argument for a virtual interface argument of a constructor or method.

Q2. Can two virtual interfaces refer to the same instance? Yes. That is the normal situation: the driver and the monitor each hold a handle to the same interface instance.

Q3. Can a virtual interface be changed during simulation? Yes. It is a variable. It can be assigned another instance of the same interface type, or `null`, at any time.

3.5 Accessing Interface Signals from Classes

Syntax

Once the handle is assigned, everything inside the interface is reached through it with the dot operator:

```
vif.en = 1;                    // write a signal
value = vif.count;             // read a signal
@(posedge vif.clk);            // wait for an edge of a signal
@(negedge vif.rst_n);
wait (vif.count == 4'd9);      // wait for a condition on a signal
vif.apply_reset(2);            // call a task of the interface
vif.drv_cb.a <= 8'h10;         // drive through a clocking block (Part 4)
@(vif.drv_cb);                 // wait for a clocking block event (Part 4)
```

Complete example: a class that drives the counter

Compile `counter_dut.sv` first, then this file.

tb_vif_basic.sv

```
class cnt_driver;

  virtual cnt_if vif;                       // handle to the counter interface

  function new(virtual cnt_if vif);
    this.vif = vif;                         // store the handle passed in by the creator
  endfunction

  task reset();
    vif.rst_n = 0;
    vif.en    = 0;
    @(negedge vif.clk);
    vif.rst_n = 1;
  endtask

  task count_for(int unsigned cycles);
    vif.en = 1;
    repeat (cycles) @(negedge vif.clk);
    vif.en = 0;
  endtask
```

```
endclass

module tb_vif_basic;

  logic clk = 0;
  always #5 clk = ~clk;

  cnt_if  cif (clk);                        // physical interface
  counter dut (cif);

  cnt_driver drv;                           // class handle (null until new)

  initial begin
    drv = new(cif);                         // the physical interface is handed to the class

    drv.reset();
    drv.count_for(3);
    $display("[%0t] count = %0d", $time, cif.count);

    drv.count_for(2);
    $display("[%0t] count = %0d (through the handle: %0d)",
             $time, cif.count, drv.vif.count);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[40] count = 3
[60] count = 5 (through the handle: 5)
```

Line by line

Line	Explanation
`virtual cnt_if vif;`	The class has a property that can refer to any instance of `cnt_if`. The class contains no testbench path and no instance name.
`function new(virtual cnt_if vif);`	The constructor takes a virtual interface as its argument.
`this.vif = vif;`	The argument and the property have the same name. `this.vif` is the property, plain `vif` is the argument (Module 4).
`vif.rst_n = 0;`	A procedural assignment to a signal of whichever instance the handle refers to.
`@(negedge vif.clk);`	The class waits for a clock edge through the handle. The clock is reachable because it is a port of the interface.
`cnt_driver drv;`	A class handle in the module. It is `null` at this point.
`drv = new(cif);`	Creates the object and passes the interface **instance** `cif` where the constructor expects a virtual interface. This is the assignment "physical to virtual".
`drv.vif.count`	From the module, the same signal can be read through the object's handle. It is the same variable as `cif.count`.

Execution flow

Time	Event
0	The object is created. `reset()` drives `rst_n` = 0 and `en` = 0.
10	Falling edge: `reset()` releases the reset and returns. `count_for(3)` sets `en` = 1.

Time	Event
15, 25, 35	Rising edges: `count` = 1, 2, 3.
40	Third falling edge: `en` = 0, `count_for` returns, the module prints `count = 3`. `count_for(2)` sets `en` = 1 again.
45, 55	Rising edges: `count` = 4, 5.
60	Second falling edge: `en` = 0, the module prints `count = 5`.

Compare this class with the module `cnt_stim` of Part 2. The code inside is nearly the same. The difference is that the class is not wired to anything: it works with whatever interface instance it is given.

What can be done through a virtual interface

Operation	Example	Notes
Read a variable or a net	`x = vif.count;`	
Write a variable	`vif.en = 1;` or `vif.en <= 1;`	Blocking or nonblocking procedural assignment
Write a net (`wire`)	**Not allowed**	A net cannot be assigned procedurally. Use the pattern of Section 5.3
Wait for an event	`@(posedge vif.clk);` `@(vif.count);`	
Call a task or function of the interface	`vif.apply_reset(2);`	
Use a clocking block	`@(vif.drv_cb); vif.drv_cb.a <= 1;`	Part 4

> **Common mistake:** Writing `vif = new();`. A virtual interface is not a class handle; there is nothing to construct. It can only be assigned an existing interface instance or another virtual interface.

> **Common mistake:** Declaring the handle with the wrong keyword order or without `virtual`: `cnt_if vif;` inside a class is an attempt to instantiate an interface, which is illegal.

Interview questions

Q1. How does a class access DUT signals? The class has a virtual interface property. The top module assigns the interface instance to it (usually through the constructor). The class then reads and writes the signals as `vif.<signal>`.

Q2. How does a class wait for a clock edge? `@(posedge vif.clk);`, or `@(vif.<clocking_block>);` when a clocking block is used.

Q3. Can a class assign a `wire` in the interface through a virtual interface? No. Procedural assignment to a net is illegal. The class assigns a `logic` variable in the interface, and a continuous assignment in the interface drives the net from that variable.

3.6 Passing Interface Handles to Classes

There are several ways to get the handle from the top module into the object that needs it. All of them are in use.

Method 1: constructor argument (used in this course)

```
class alu_driver;
  virtual alu_if.DRV vif;
  function new(virtual alu_if.DRV vif);
    this.vif = vif;
  endfunction
endclass

class alu_env;
  virtual alu_if vif;
  alu_driver     drv;
  alu_monitor    mon;

  function new(virtual alu_if vif);
    this.vif = vif;
    drv = new(vif);          // the environment passes the handle down
    mon = new(vif);
  endfunction
endclass

// in tb_top:
//   alu_env env;
//   initial begin  env = new(aif);  ...  end
```

The object cannot exist without a handle, so it is hard to forget.

Method 2: assign the property directly after construction

```
drv     = new();
drv.vif = aif;             // set from tb_top, or from the environment
```

Simple, but nothing forces the assignment to happen, and the property must be public.

Method 3: a set method

```
class alu_monitor;
  protected virtual alu_if.MON vif;
  function void set_vif(virtual alu_if.MON vif);
    this.vif = vif;
  endfunction
endclass
```

Keeps the property protected and gives one place for checks.

Method 4: a configuration object

```
class alu_config;
  virtual alu_if vif;
  int unsigned   num_txns = 20;
endclass
```

One object carries the handle together with the other settings of the testbench. Every component receives the configuration object and takes `cfg.vif` from it. When a second interface is added, only the configuration class changes, not every constructor.

Method 5: a central database (UVM, awareness only)

```
// in tb_top:
uvm_config_db #(virtual alu_if)::set(null, "*", "vif", aif);

// in the driver's build_phase:
if (!uvm_config_db #(virtual alu_if)::get(this, "", "vif", vif))
  `uvm_fatal("NOVIF", "virtual interface not set")
```

The top module stores the handle under a name; any component fetches it by that name. No constructor has to pass it along. Module 11 introduces UVM.

Comparison

Method	Advantage	Disadvantage
Constructor argument	The object is never without its handle	Every level between top and component must pass it on
Direct assignment	Shortest	Easy to forget; exposes the property
Set method	Encapsulation; a place for checks	Easy to forget to call
Configuration object	Scales to many interfaces and settings	One more class
`uvm_config_db`	No passing through intermediate levels	UVM only; a wrong string name is found only at run time

Interview questions

Q1. How do you pass an interface to a class? Declare a virtual interface property in the class and give it a value from outside: most commonly as a constructor argument, otherwise by direct assignment, a set method, a configuration object, or in UVM through `uvm_config_db`.

Q2. How does the virtual interface travel from the top module to the driver in a layered testbench? The top module passes the interface instance to the test (or environment) constructor. The environment stores it and passes it to the constructors of the driver and the monitor. Each component keeps its own copy of the handle; all copies refer to the same instance.

Q3. Why is a virtual interface not passed through a mailbox like a transaction? It could be, since it is a variable, but it is configuration, not data. It is set once when the testbench is built, before any component runs.

3.7 The Null Virtual Interface Error

What happens

A virtual interface that has not been assigned is `null`. Using it to read a signal, write a signal or wait for an event is a fatal **run-time** error: the simulator reports a null virtual interface (null pointer) dereference with the file and line number, and stops. The compiler cannot detect it, because the assignment happens at run time.

```
class cnt_driver;
  virtual cnt_if vif;
  task run();
    vif.en = 1;            // run-time error if vif was never assigned
  endtask
endclass

module tb;
  cnt_driver drv;
  initial begin
    drv = new();           // nobody set drv.vif
    drv.run();             // fatal error here
  end
endmodule
```

Typical causes

Cause	Fix
The handle was never passed in	Pass it through the constructor so that it cannot be forgotten
The component started running before the handle was set	Build and connect everything first, then start `run()`
The constructor argument has the same name as the property and `this.` was forgotten: `vif = vif;`	Write `this.vif = vif;`
The environment received the handle but did not pass it on to a sub-component	Pass it in every `new(...)` call
A different object was configured than the one that runs	Check that the handle of the object in use is the one that was set

Defensive check

Compile `counter_dut.sv` first, then this file.

tb_vif_null.sv

```
class cnt_user;

  virtual cnt_if vif;

  function void set_vif(virtual cnt_if vif);
    this.vif = vif;
  endfunction

  function bit is_connected();
    return (vif != null);
  endfunction

  task enable();
    if (vif == null) begin
      $display("[%0t] ERROR: vif is null, cannot drive", $time);
      return;
    end
    vif.en = 1;
    $display("[%0t] en driven to 1", $time);
  endtask

endclass

module tb_vif_null;

  logic clk = 0;
  always #5 clk = ~clk;

  cnt_if  cif (clk);
  counter dut (cif);

  cnt_user u;

  initial begin
    u = new();
    $display("connected = %b", u.is_connected());
    u.enable();

    u.set_vif(cif);
    $display("connected = %b", u.is_connected());
    u.enable();
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
connected = 0
[0] ERROR: vif is null, cannot drive
connected = 1
[0] en driven to 1
```

Line by line

Line	Explanation
`return (vif != null);`	A virtual interface can be compared with `null`, like a class handle.
`if (vif == null) begin ... return; end`	The task checks the handle before using it and reports a clear message instead of letting the simulator crash. In a real testbench this would be `$fatal`.
`u = new();`	The object exists but its `vif` is still `null`.
`u.set_vif(cif);`	Now the handle refers to `cif`, and the second call to `enable()` drives the signal.

> **Interview trap:** "Your simulation stops at time 0 with a null object access in the driver. What do you check?" Two handles can be null: the class handle of the driver itself (nobody called `new`), and the virtual interface inside the driver (nobody assigned it). The message normally names the line; if the line is `vif.something`, the virtual interface was not connected. The classic root cause is `vif = vif;` in the constructor, without `this.`.

Interview questions

Q1. What happens when a null virtual interface is used? A fatal run-time error (null handle dereference) at the statement that uses it. It is not caught at compile time.

Q2. How do you avoid it? Pass the interface through the constructor, connect all handles before starting any component, and check `vif != null` (with `$fatal` on failure) at the beginning of the component's `run()` task.

3.8 Arrays of Virtual Interfaces

In plain English

Section 2.7 showed that an array of interface **instances** cannot be indexed with a variable. A virtual interface is an ordinary variable, so an **array of virtual interfaces** is an ordinary array: it can be indexed with a variable, looped over with `foreach`, and passed to a class. The usual pattern is to copy the instances into such an array once, and use the array from then on.

Syntax

```
cnt_if         cif  [2] (clk);     // array of instances  (static, constant index)
virtual cnt_if vifs [2];           // array of handles    (ordinary variables)

initial begin
  vifs[0] = cif[0];                // constant indices on the right-hand side
  vifs[1] = cif[1];
end
```

For many elements, a generate loop does the copying:

```
for (genvar i = 0; i < 2; i++) begin : g_vif
  initial vifs[i] = cif[i];        // i is a constant inside each generated block
end
```

Complete example

Compile `counter_dut.sv` first, then this file.

tb_vif_array.sv

```
module tb_vif_array;

  logic clk = 0;
  always #5 clk = ~clk;

  cnt_if cif [2] (clk);

  for (genvar i = 0; i < 2; i++) begin : g_dut
    counter dut (cif[i]);
  end

  virtual cnt_if vifs [2];                // one handle per interface instance

  initial begin
    vifs[0] = cif[0];
    vifs[1] = cif[1];

    foreach (vifs[i]) begin               // a variable index is fine on the handles
      vifs[i].rst_n = 0;
      vifs[i].en    = 0;
    end
    @(negedge clk);
    foreach (vifs[i]) vifs[i].rst_n = 1;

    vifs[0].en = 1;
    repeat (3) @(negedge clk);
    vifs[1].en = 1;
    repeat (2) @(negedge clk);

    foreach (vifs[i])
      $display("[%0t] counter %0d = %0d", $time, i, vifs[i].count);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[60] counter 0 = 5
[60] counter 1 = 2
```

Line by line

Line	Explanation
`virtual cnt_if vifs [2];`	A fixed-size array of two handles. Both start as `null`.
`vifs[0] = cif[0]; vifs[1] = cif[1];`	Each handle is pointed at one instance. The indices on `cif` are constants, as required.
`foreach (vifs[i]) vifs[i].rst_n = 0;`	The same statement is applied to every bus. This loop would be illegal on `cif[i]` directly.
`vifs[i].count`	Reads the counter of bus `i`.

Execution flow

Time	Counter 0	Counter 1
0 to 10	Reset	Reset
10	Reset released; `en` = 1	Reset released; `en` = 0
15, 25, 35	1, 2, 3	0
40		`en` = 1

Time	Counter 0	Counter 1
45, 55	4, 5	1, 2
60	Printed: 5	Printed: 2

> **Verification application:** An environment for a design with N identical ports holds `virtual port_if vifs[N]` and an array of N agents. In a loop it creates agent `i` and gives it `vifs[i]`. Nothing in the agent, driver or monitor classes knows how many ports exist.

Interview questions

Q1. How do you drive several instances of an interface from one loop? Declare an array of virtual interfaces, assign each element one interface instance (with constant indices or in a generate loop), and loop over the array of handles.

Q2. Why can an array of virtual interfaces be indexed with a variable when an array of interfaces cannot? An array of interfaces is a set of hierarchy instances, and hierarchical names need constant indices. An array of virtual interfaces is an array of variables, like an array of class handles.

3.9 Virtual Interfaces with Modports and Parameters

With a modport

```
class alu_monitor;
  virtual alu_if.MON vif;          // this class can only do what modport MON allows
endclass
```

Declaration	Access through the handle
`virtual alu_if vif;`	Everything in the interface, in both directions
`virtual alu_if.MON vif;`	Only the items listed in modport `MON`, in the listed directions. Anything else is a compile error

Using modport-restricted handles in the driver and the monitor gives the class-based testbench the same protection that Part 2 gave to modules.

With parameters

```
mem_if #(.AW(16), .DW(32)) mif (clk);

virtual mem_if #(.AW(16), .DW(32)) vif_ok  = mif;     // types match
// virtual mem_if                  vif_bad = mif;     // ILLEGAL: default parameters differ
```

Each set of parameter values is a separate type, and a handle can only refer to instances of exactly its own type. A class that must work with any width is itself parameterised and passes its parameters to the handle type:

```
class mem_driver #(int AW = 4, int DW = 8);
  virtual mem_if #(AW, DW) vif;
endclass

mem_driver #(16, 32) drv;       // works with the 16/32 instance
```

Parameterised classes are covered in Module 11.

Preview: the shape of the ALU driver

Part 4 completes this class with clocking blocks. The structure is already fixed by what you know:

```
class alu_driver;

  virtual alu_if.DRV vif;                     // 1. the handle

  function new(virtual alu_if.DRV vif);       // 2. received from outside
    this.vif = vif;
  endfunction

  task drive(bit [7:0] a, bit [7:0] b, alu_op_e op);
    // 3. pin-level activity, always through vif
  endtask

endclass
```

> **Common mistake:** Declaring the handle without the modport in the class (`virtual alu_if vif;`) "to keep it simple". The code works, but the monitor can then drive signals and the driver can read signals it should not. Use the modport that matches the role.

Interview questions

Q1. What is the benefit of `virtual alu_if.DRV vif` over `virtual alu_if vif`? The handle is restricted to the driver's modport. The driver cannot write a DUT output or bypass the clocking block; a mistake is reported at compile time.

Q2. An interface is parameterised. What must be true of the virtual interface declaration? It must specify the same parameter values as the instance it will refer to, because differently parameterised interfaces are different types.

3.10 Summary of Part 3

Step	Where	Code
1. Declare the interface	A file compiled before the classes	`interface cnt_if (input logic clk); ... endinterface`
2. Instantiate it and connect the DUT	Top module	`cnt_if cif (clk); counter dut (cif);`
3. Declare a handle in the class	Class	`virtual cnt_if vif;`
4. Receive the handle	Class constructor	`function new(virtual cnt_if vif); this.vif = vif; endfunction`

Step	Where	Code
5. Hand the instance to the class	Top module	`drv = new(cif);`
6. Use the signals	Class methods	`vif.en = 1; @(posedge vif.clk); x = vif.count;`

> **How to say it in an interview:** "A class cannot have ports and cannot instantiate an interface, because classes are dynamic and interfaces are static. So the class holds a virtual interface, which is a handle to an interface instance. The top module creates the real interface, connects it to the DUT, and passes it to the testbench classes, usually through their constructors. Inside the class, signals are accessed as `vif.signal`. If the handle is never assigned it is null, and using it gives a run-time error."

Part 4: Clocking Blocks

4.1 The Problem: Races Between the Testbench and the DUT

In plain English

A synchronous design reads its inputs at the rising edge of the clock. A testbench that changes those inputs **at the same rising edge** creates a question with no defined answer: does the design see the old value or the new one? Both the design and the testbench are waiting for the same edge. When it arrives, the simulator runs one of them first, and the language does not say which. The result depends on the simulator, on the order of the files, sometimes on an unrelated edit elsewhere in the code. This is a **race condition**.

In real hardware the question does not arise, because the previous flip-flop needs some time to change its output after the clock edge (clock-to-output delay), and the receiving flip-flop has long since captured the old value. A zero-delay RTL simulation has no such delay, so the testbench has to create that separation itself.

A racy testbench

The design is a single flip-flop.

dff.sv

```
module dff (input logic clk, input logic d, output logic q);
  always_ff @(posedge clk) q <= d;
endmodule
```

Compile `dff.sv` first, then this file.

tb_race.sv

```
module tb_race;

  logic clk = 0, d = 0, q;
  always #5 clk = ~clk;

  dff dut (.*);

  initial begin
    @(posedge clk);          // time 5: wake up at the rising edge
    d = 1;                   // change the DUT input at that same edge (blocking)
    @(posedge clk);          // time 15
    $display("[%0t] q = %b", $time, q);
    $finish;
  end

endmodule
```

There is no single expected output. At time 5 two processes are woken by the same edge: the `always_ff` block in the DUT and the `initial` block in the testbench.

If the simulator runs...	At time 5 the DUT samples	`q` after time 5	Printed at time 15
the DUT's `always_ff` first	`d` = 0 (old value)	0	`q = 0`
the testbench's `initial` first	`d` = 1 (new value)	1	`q = 1`

Both are legal behaviours of a standard-compliant simulator. (At time 15 the DUT samples `d` = 1 in both cases, but that update of `q` is a nonblocking assignment and has not taken effect yet when `$display` runs.)

The two races

Race	What happens	Consequence
Drive race	The testbench changes a DUT input in the same time step in which the DUT samples it	The DUT may capture the new value one cycle too early
Sample race	The testbench reads a DUT output in the same time step in which the DUT changes it	The testbench may read the value from before or after the edge

For a pure RTL design whose outputs come from nonblocking assignments, the sample race is milder, because the outputs change only after all sampling in the Active region is done (next section). It becomes real with behavioural models that use blocking assignments, with zero-delay gate-level netlists, and when the clock reaches the testbench and the DUT through different paths.

Ways to avoid the race

Technique	How it separates drive from sample	Weakness
Drive on the opposite (falling) edge	Half a cycle of separation	Wrong timing for protocols that use both edges; hand-written everywhere
Drive with a small delay after the edge: `@(posedge clk); #1 d = 1;`	The DUT has sampled before the delay ends	Delays scattered through the code; must be smaller than the clock period
Drive with a nonblocking assignment: `@(posedge clk); d <= 1;`	The update happens in the NBA region, after the DUT has sampled	Solves driving only; easy to break by one `=` somewhere
Clocking block	Sampling and driving times are declared once, in one place, and applied automatically	None in practice; this is the standard solution

Parts 1 to 3 used the first technique. The rest of this part explains the last one.

Interview questions

Q1. What is a race condition between the testbench and the DUT? The testbench drives or samples a signal in the same simulation time step in which the DUT samples or drives it, so the result depends on the order in which the simulator happens to execute the two processes. The simulation result is then not deterministic.

Q2. Why does the race not exist in real hardware? Real flip-flops have a clock-to-output delay, and data arrives through logic with delay, so an input changes after the capturing edge, not exactly at it. RTL simulation has zero delay and loses this separation.

Q3. Give three ways to avoid a testbench-DUT race. Drive on the inactive clock edge; drive with nonblocking assignments or after a small delay; use a clocking block (optionally together with a program block).

4.2 How the Simulator Orders Events in One Time Step

To understand exactly what a clocking block does, you need a simplified picture of the **scheduler**. Everything that happens at one value of simulation time (one *time step* or *time slot*) is processed in ordered **regions**.

```
one time step, for example t = 25 (a rising clock edge)

+-----------+  values are still those from the end of the previous time step
| Preponed  |  -> clocking block inputs with skew #1step are sampled here
+-----------+     (concurrent assertions also sample here)
      |
+-----------+  module code runs: blocking assignments, continuous assignments,
|  Active   |  evaluation of the right-hand side of nonblocking assignments.
+-----------+  The DUT's always_ff blocks READ their inputs here.
      |
+-----------+
| Inactive  |  statements delayed with #0
+-----------+
      |
+-----------+  nonblocking assignments update their left-hand sides.
|    NBA    |  The DUT's flip-flop OUTPUTS change here.
+-----------+
      |
+-----------+  concurrent assertions are evaluated;
| Observed  |  clocking block inputs with skew #0 are sampled;
+-----------+  the clocking block's own event (@(cb)) is triggered
      |
+-----------+
| Reactive  |  program block code and assertion action blocks run
+-----------+
      |
+-----------+
|  Re-NBA   |  clocking block outputs with skew #0 are driven here
+-----------+
      |
+-----------+
| Postponed |  $monitor and $strobe print; nothing can change any more
+-----------+

If a later region schedules new activity for an earlier region, the simulator loops
back (for example Re-NBA -> Active) until the time step is quiet, then advances time.
```

Three facts from this picture explain everything else in Part 4:

1. **The DUT reads in Active and writes in NBA.** At a clock edge, flip-flops sample their inputs in the Active region and their outputs change in the NBA region.
2. **Preponed is before everything.** A value sampled in the Preponed region is the value the signal had just before the clock edge, which is exactly what a flip-flop clocked by that edge captures.
3. **Re-NBA and "edge + delay" are after the DUT has sampled.** A drive made there cannot be seen by the DUT at this edge. It is seen at the next edge.

Interview questions

Q1. In which region are nonblocking assignments updated? In which region are blocking assignments executed? Blocking assignments execute in the Active region. The right-hand side of a nonblocking assignment is evaluated in the Active region and its left-hand side is updated in the NBA region.

Q2. What is the Preponed region used for? For sampling. Clocking block inputs with `#1step` skew and concurrent assertions take their values there, before any activity of the time step, so they see the stable values from just before the clock edge.

Q3. In which region does program block code run? In the Reactive region, after the design has finished its Active and NBA processing for that time step.

4.3 What Is a Clocking Block?

In plain English

A clocking block is a statement of timing discipline. It says: "these signals are synchronous to this clock edge; whenever I read one of them, give me the value it had just before the edge; whenever I write one of them, apply the value a little after the edge". After that, the testbench code just reads and writes through the clocking block and never thinks about races again.

Technical definition

A `clocking` block, declared between `clocking` and `endclocking`, groups signals that are synchronous to a particular **clocking event** (such as `posedge clk`) and specifies, for each one, a direction and a **skew**:

- an **input** is *sampled* by the clocking block at the clocking event minus the input skew;
- an **output** is *driven* by the clocking block at the clocking event plus the output skew.

The direction is from the testbench's point of view: an input of the clocking block is something the testbench reads (a DUT output); an output is something the testbench drives (a DUT input).

Why it is required

Need	How the clocking block meets it
No drive race	Outputs change after the clock edge, when the DUT has already sampled
No sample race	Inputs are captured before the clock edge, when nothing is changing
Cycle-accurate testbench code	Code waits for `@(cb)` and thinks in clock cycles, not in time units
One place for the timing	Skews are declared once in the interface instead of `#1` delays spread over the driver
Works with any DUT timing	RTL, gate-level with delays: only the skew values change, not the testbench code
Models setup and hold	The input skew corresponds to sampling "setup time before the edge"; the output skew to "clock-to-output delay after the edge"

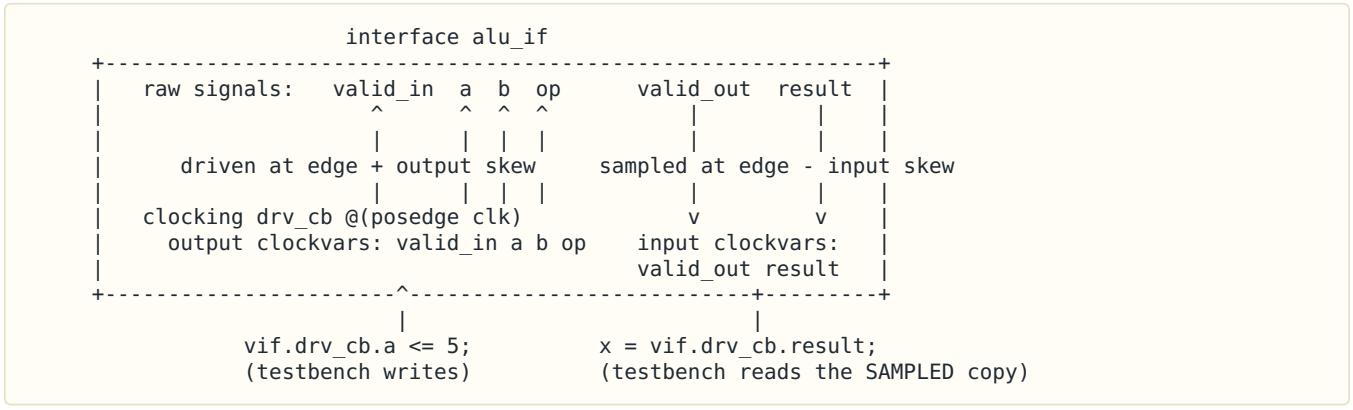
Syntax

```
clocking <name> @(<clocking event>);
  default input <input skew> output <output skew>;   // optional; applies to all signals
  input  <signals>;                                  // sampled by the testbench
  output <signals>;                                  // driven by the testbench
  inout  <signals>;                                  // both
  input  #<skew> <signal>;                           // per-signal skew overrides default
  output negedge <signal>;                           // a skew can also be an edge
endclocking
```

```
// Example: the driver's clocking block of the ALU interface
clocking drv_cb @(posedge clk);
  default input #1step output #1;
  output valid_in, a, b, op;         // the driver drives the DUT inputs
  input  valid_out, result;          // and may look at the DUT outputs
endclocking
```

- The names inside the clocking block refer to signals of the same name in the enclosing interface (or module). The clocking block's own versions are called **clockvars** and are accessed as `<clocking_block>.<signal>`, for example `drv_cb.a`.
- A clocking block may be declared in a module, an interface, a program or a checker. It may not be declared in a class or a package. In verification it is almost always inside the interface.
- A signal may appear in several clocking blocks (the ALU inputs appear in both `drv_cb` and `mon_cb`).
- Clocking blocks are simulation-only. They are ignored by synthesis.

How it works internally



- For every **input**, the clocking block keeps a sampled copy. At each clocking event the copy is refreshed with the value of the raw signal at "edge minus input skew". Reading `cb.sig` returns the copy, not the live signal. The copy stays constant until the next clocking event.
- For every **output**, a write `cb.sig <= value` does not change the raw signal immediately. The clocking block applies it to the raw signal at "edge plus output skew".

Interview questions

Q1. What is a clocking block? A construct that groups signals synchronous to a clock and defines when the testbench samples them (input skew before the clock event) and when it drives them (output skew after the clock event).

Q2. Why are clocking blocks needed? To remove races between the testbench and the DUT and to let the testbench work in terms of clock cycles. Sampling happens before the edge and driving after it, so the order in which the simulator executes processes at the edge no longer matters.

Q3. From whose point of view are the directions in a clocking block? The testbench's. `input` means the testbench samples it; `output` means the testbench drives it.

Q4. Where can a clocking block be declared? In an interface, module, program or checker. Not in a class or a package.

Q5. Is a clocking block synthesizable? No. It is a testbench construct.

4.4 Input and Output Skews

Definition

A **skew** is the distance in time between the clocking event and the moment a signal is sampled or driven.

```
                input skew                output skew
             <------------->            <------------->
                           |            |
      sample point         | clock edge |         drive point
           v               v            |              v
---------+---------------+------------+--------------+----------> time
                         ^
                  the clocking event (for example posedge clk)

  Inputs are sampled BEFORE the edge.      Outputs are driven AFTER the edge.
```

Skew values

Skew	For an input: sampled...	For an output: driven...
`#1step`	In the Preponed region of the clock edge's time step: the last value before the edge	Not used for outputs
`#0`	In the **Observed** region of the clock edge's time step: **after** the DUT's NBA updates	In the Re-NBA region of the clock edge's time step
`#N` (a number, for example `#2`)	N time units before the clock edge	N time units after the clock edge
`#2ns`	A time literal may be used	
`posedge` / `negedge`	At that edge of the clock	At that edge of the clock

The defaults

If a clocking block gives no skew, the language uses:

Direction	Default skew	Meaning
`input`	`#1step`	Sample the value from just before the clock edge
`output`	`#0`	Drive in the Re-NBA region of the same time step as the clock edge

`#1step` is the smallest possible step back: one unit of the simulator's global time precision before the clock edge, which is the same as the value in the Preponed region. It is the right choice for inputs in almost every testbench.

The ALU interface uses `default input #1step output #1;`. The explicit output skew of one time unit makes the driven signals change visibly after the edge in the waveform, which is easier to debug than a change at exactly the edge.

The #0 input skew trap

`input #0` does **not** mean "the value at the edge as the DUT sees it". It means "sample in the Observed region", which is after the DUT's flip-flops have already updated. The testbench then sees the **new** value of a DUT output, one cycle earlier than a flip-flop clocked by the same edge would.

Complete example: #1step versus #0

tb_cb_skew.sv

```
module tb_cb_skew;

  logic       clk = 0;
  logic [3:0] count = 0;

  always #5 clk = ~clk;

  // A stand-in for the DUT: a register that changes at every rising edge
  always @(posedge clk) count <= count + 1'b1;

  clocking cb_step @(posedge clk);
    input #1step count;          // the value just before the edge
  endclocking

  clocking cb_zero @(posedge clk);
    input #0 count;              // the value after the edge has been processed
  endclocking

  initial begin
    repeat (3) begin
      @(negedge clk);            // look at the samples in the quiet middle of the cycle
      $display("[%0t] count now = %0d   #1step sample = %0d   #0 sample = %0d",
               $time, count, cb_step.count, cb_zero.count);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[10] count now = 1   #1step sample = 0   #0 sample = 1
[20] count now = 2   #1step sample = 1   #0 sample = 2
[30] count now = 3   #1step sample = 2   #0 sample = 3
```

Line by line

Line	Explanation
`always @(posedge clk) count <= count + 1'b1;`	The register is updated by a nonblocking assignment, so it changes in the NBA region of each rising edge: to 1 at time 5, 2 at time 15, 3 at time 25.
`clocking cb_step ... input #1step count;`	A clocking block declared directly in a module. Its copy of `count` is taken in the Preponed region.
`clocking cb_zero ... input #0 count;`	The same signal with an explicit zero skew: its copy is taken in the Observed region.
`@(negedge clk);`	The samples are printed at the falling edge, when nothing is changing, so the printout itself has no race.
`cb_step.count`, `cb_zero.count`	The sampled copies held by the two clocking blocks. They keep their value until the next rising edge.

What happened at the rising edge at time 15

Region	Event
Preponed	`cb_step` samples `count` = 1 (the value before the edge)
Active	The `always` block evaluates `count + 1` = 2
NBA	`count` becomes 2

Region	Event
Observed	`cb_zero` samples `count` = 2 (the value after the edge)

The `#1step` sample is what another flip-flop would have captured at that edge. The `#0` sample is already one step ahead.

> **Interview trap:** "What are the default input and output skews of a clocking block?" Input `#1step`, output `#0`. A common wrong answer is "`#0` for both".

> **Interview trap:** "What is the difference between `input #0` and `input #1step`?" `#1step` samples in the Preponed region, before the clock edge is processed: the old, stable value. `#0` samples in the Observed region, after the design's nonblocking updates: the new value. A monitor with `#0` reports DUT outputs one cycle earlier than the DUT's neighbours would see them, which shifts the scoreboard's comparison by a cycle.

Interview questions

Q1. What is skew in a clocking block? The offset between the clocking event and the point at which a signal is sampled (input skew, before the event) or driven (output skew, after the event).

Q2. What does `#1step` mean? Sample one time step before the clocking event, that is, the value of the signal in the Preponed region of the clock edge's time step: the last stable value before the edge.

Q3. What does `default input #2 output #3;` mean in a clocking block with `@(posedge clk)`? Inputs are sampled 2 time units before each rising clock edge, and outputs are driven 3 time units after it.

Q4. Can each signal have its own skew? Yes. A skew written in a signal's own declaration (`input #4 data;`) overrides the default of the clocking block.

4.5 Sampling and Driving Through a Clocking Block

The four operations

Operation	Syntax	Meaning
Wait for the clocking event	`@(cb);` or `@(vif.cb);`	Suspend until the next clock event of this clocking block. On waking, the input samples for that event are available
Read an input	`x = cb.sig;`	Returns the value sampled at the most recent clocking event
Drive an output	`cb.sig <= value;`	A **synchronous drive**: the value is applied to the signal at the clocking event plus the output skew
Wait for a sampled value to change	`@(cb.sig);` `@(posedge cb.sig);`	Wakes when the sampled copy changes, which can only happen at a clocking event

Rules

Rule	Example	Result
Outputs are driven only with `<=`	`cb.a <= 8'h10;`	Legal
Blocking assignment to a clockvar	`cb.a = 8'h10;`	**Compile error**
Reading an output-only clockvar	`x = cb.a;` (when `a` is an `output`)	**Compile error**. Declare it `inout` if it must also be read
Writing an input clockvar	`cb.result <= 0;`	**Compile error**
A drive executed exactly at a clocking event	After `@(cb);`	Takes effect at that event plus the output skew
A drive executed at any other time	After `#3;` or `@(negedge clk);`	Takes effect as if it had been executed at the **next** clocking event
Several drives to the same output for the same event	Two `cb.a <= ...` statements in one cycle	The last one wins
The drive statement does not block	`cb.a <= 1;`	The process continues at once; the signal changes later

The `<=` used for a synchronous drive looks like a nonblocking assignment but its timing is defined by the clocking block: clock event plus output skew.

Do not mix raw clock edges with clocking block samples

```
// Risky
@(posedge vif.clk);
x = vif.mon_cb.result;      // has the clocking block refreshed its sample yet?

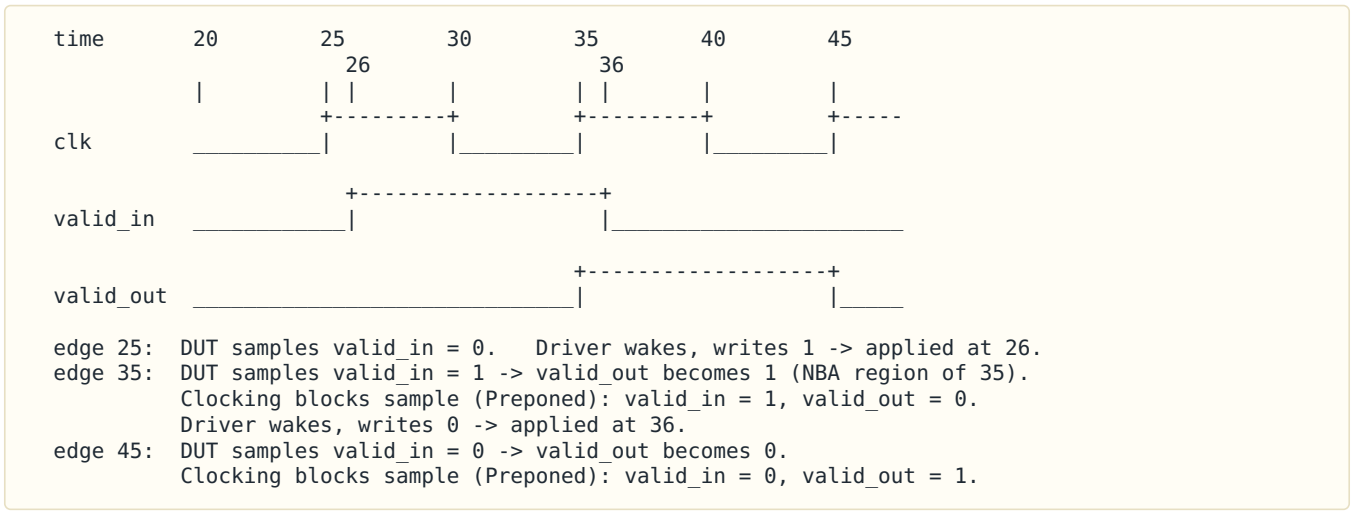
// Correct
@(vif.mon_cb);
x = vif.mon_cb.result;      // guaranteed to be the sample taken at this event
```

A process that waits on the raw signal `@(posedge clk)` and then reads a clockvar is racing against the clocking block's own update of that clockvar: it may get the sample of this edge or of the previous one. A process that waits on the clocking block itself, `@(cb)`, is guaranteed by the language to see the updated samples.

> **Key point:** Once a signal is behind a clocking block, use the clocking block for everything: wait with `@(vif.cb)`, read `vif.cb.sig`, drive `vif.cb.sig <= value`. Do not go around it with `@(posedge vif.clk)` or `vif.sig`.

Timing of one driven value and one sampled value

The diagram shows the ALU with a clock period of 10, rising edges at 25, 35 and 45, and the driver's clocking block `default input #1step output #1`. The driver executes `vif.drv_cb.valid_in <= 1` when it wakes at edge 25 and `vif.drv_cb.valid_in <= 0` when it wakes at edge 35.



Read the diagram as a rule: **a value driven through the clocking block in one cycle is seen by the DUT at the next rising edge, and a value produced by the DUT at one rising edge is seen through the clocking block at the next rising edge.** That is exactly how one flip-flop sees another, so the testbench behaves like a synchronous neighbour of the DUT.

Interview questions

Q1. How do you drive a signal through a clocking block? With the synchronous drive `cb.signal <= value;`. Only the `<=` form is allowed. The value is applied at the clocking event plus the output skew.

Q2. How do you sample a signal through a clocking block? Wait for the clocking event with `@(cb);` and read `cb.signal`, which returns the value captured at the input skew before that event.

Q3. What does `@(vif.drv_cb)` wait for? The next occurrence of the clocking event of `drv_cb`, for example the next rising edge of `clk`. On waking, the block's input samples for that edge are valid.

Q4. What happens if `cb.a <= 1;` is executed between two clock edges? The drive is treated as if it were executed at the next clocking event, so the signal changes at that event plus the output skew.

Q5. Why should you not write `@(posedge vif.clk)` and then read `vif.cb.sig`? There is a race between that process and the clocking block's update of its sampled values, so the read may return the previous sample. Waiting on `@(vif.cb)` guarantees the samples are current.

4.6 Race Prevention with a Clocking Block

This is the racy flip-flop testbench of Section 4.1, rewritten with an interface and a clocking block.

Compile `dff.sv` first, then this file.

tb_race_fixed.sv

```
interface dff_if (input logic clk);
  logic d = 0;
  logic q;

  clocking cb @(posedge clk);
    default input #1step output #1;
    output d;                 // the testbench drives d
    input  q;                 // the testbench samples q
  endclocking
endinterface

module tb_race_fixed;

  logic clk = 0;
  always #5 clk = ~clk;

  dff_if dif (clk);
  dff    dut (.clk(clk), .d(dif.d), .q(dif.q));

  initial begin
    @(dif.cb);                // time 5
    dif.cb.d <= 1;            // applied at time 6: after the DUT has sampled
    @(dif.cb);                // time 15
    $display("[%0t] sampled q = %b", $time, dif.cb.q);
    @(dif.cb);                // time 25
    $display("[%0t] sampled q = %b", $time, dif.cb.q);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[15] sampled q = 0
[25] sampled q = 1
```

Line by line

Line	Explanation
`logic d = 0;`	`d` starts at 0 instead of X, so the first sampled value is known.
`clocking cb @(posedge clk);`	All timing is relative to the rising edge of `clk`.
`default input #1step output #1;`	Sample just before the edge; drive one time unit after it.
`output d; input q;`	From the testbench's side: it drives `d` and observes `q`.
`@(dif.cb);`	Wait for the clocking event. The module accesses the clocking block as `<interface instance>.<clocking block>`.
`dif.cb.d <= 1;`	Synchronous drive. Executed at time 5, applied to `dif.d` at time 6.
`dif.cb.q`	The sampled copy of `q`.

Execution flow

Time	Region	Event
5	Active	The DUT samples `d` = 0 and schedules `q` = 0. The testbench wakes and requests `d` = 1.
5	NBA	`q` becomes 0.
6		The clocking block applies `d` = 1.

Time	Region	Event
15	Preponed	`cb` samples `q` = 0.
15	Active / NBA	The DUT samples `d` = 1; `q` becomes 1.
15		The testbench wakes and prints the **sampled** `q` = 0.
25	Preponed	`cb` samples `q` = 1.
25		The testbench prints the sampled `q` = 1.

The output is the same on every simulator, whatever order the processes are executed in, because the DUT's sampling (Active region of the edge), the testbench's sampling (Preponed region of the edge) and the testbench's driving (one time unit later) never coincide.

Racy and fixed versions side by side

	Racy version (`tb_race.sv`)	Fixed version (`tb_race_fixed.sv`)
Wait for the clock	`@(posedge clk);`	`@(dif.cb);`
Drive	`d = 1;` at the edge, in the Active region	`dif.cb.d <= 1;` applied at edge + 1
Sample	`q` read directly at the edge	`dif.cb.q`, captured before the edge
DUT sees the new `d`	At this edge or the next, depending on the simulator	Always at the next edge
Result	Simulator-dependent	Deterministic

Interview questions

Q1. How does a clocking block prevent race conditions? It separates the three activities in time. The DUT samples its inputs in the Active region of the clock edge. The clocking block samples DUT outputs before that, in the Preponed region. It drives DUT inputs after that, at the edge plus the output skew (or in the Re-NBA region for zero skew). Because they never happen at the same point, the execution order chosen by the simulator cannot change the result.

Q2. Is a clocking block the only way to avoid races? No. Driving on the opposite clock edge, driving with nonblocking assignments, adding a small delay, or putting the testbench in a program block also work. The clocking block is the most complete: it covers both sampling and driving, keeps the timing in one place, and works unchanged for gate-level simulation.

4.7 Cycle Delays (##N) and Default Clocking

Definition

Inside a module, interface or program, one clocking block can be made the **default clocking**. The cycle delay operator `##N` then means "wait N clocking events of the default clocking block".

```
default clocking cb @(posedge clk);     // declares a clocking block, makes it the default
  ...
endclocking

default clocking cb;                     // or: makes an existing clocking block the default

##1;                    // wait for 1 clocking event
##5;                    // wait for 5 clocking events
##(n + 1);              // an expression needs parentheses
cb.sig <= ##2 value;    // drive 'value' two clocking events from now (does not block)
```

Form	Meaning
`##N;` as a statement	Blocks the process for N clocking events. Equivalent to `repeat (N) @(cb);`
`cb.sig <= ##N value;`	The value is evaluated now and driven N clocking events later (plus the output skew). The process is not blocked
`##N` inside a sequence or property	Delay of N cycles in an assertion (Module 10). The same symbol, a related meaning

Complete example

tb_cycle_delay.sv

```
module tb_cycle_delay;

  logic       clk = 0;
  logic [3:0] d   = 0;

  always #5 clk = ~clk;

  default clocking cb @(posedge clk);
    default input #1step output #2;
    output d;
  endclocking

  always @(d) $display("[%0t] d changed to %0d", $time, d);

  initial begin
    ##1;                            // wait for 1 clocking event: time 5
    $display("[%0t] after ##1", $time);
    cb.d <= 4'd7;                   // applied at 5 + 2 = 7

    ##3;                            // three more events: 15, 25, 35
    $display("[%0t] after ##3", $time);
    cb.d <= ##2 4'd9;               // two events later (55), plus skew: 57

    ##3;                            // 45, 55, 65
    $display("[%0t] done", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[5] after ##1
[7] d changed to 7
[35] after ##3
[57] d changed to 9
[65] done
```

Line by line

Line	Explanation
`default clocking cb @(posedge clk);`	Declares `cb` and makes it the default clocking of this module.
`always @(d) $display(...)`	Reports every change of the raw signal `d`, to show when the drives really happen.
`##1;`	Executed at time 0. Waits for the next clocking event, the rising edge at time 5.
`cb.d <= 4'd7;`	Executed at the clocking event of time 5. With output skew `#2`, `d` changes at time 7.
`##3;`	Waits for the events at 15, 25 and 35.
`cb.d <= ##2 4'd9;`	Executed at 35. The drive is scheduled two events later (55) plus the skew: time 57. The statement does not wait; execution continues immediately.
`##3;`	Waits for 45, 55 and 65. The scheduled change at 57 is printed while this wait is in progress.

The limitation in classes

`##N` needs a default clocking in the scope where it is written. A class, and a package, has no default clocking, so `##2;` inside a class method is a compile error. In class-based testbenches the equivalent is:

```
repeat (2) @(vif.drv_cb);
```

> **Interview trap:** "Can you use `##5` in a driver class?" No. There is no default clocking block in a class. Use `repeat (5) @(vif.cb);`.

Interview questions

Q1. What does `##3` mean? As a procedural statement it waits for three clocking events of the default clocking block of the current module, interface or program.

Q2. What is needed for `##N` to compile as a statement? A `default clocking` declaration in the enclosing module, interface or program.

Q3. What is the difference between `#5` and `##5`? `#5` is a delay of 5 time units. `##5` is a delay of 5 clock cycles (clocking events).

Part 5: Putting It Together

5.1 The Final ALU Interface

This is the interface used for the rest of the course. Module 9 builds the layered testbench on it without changing a line.

alu_if.sv

```
interface alu_if (input logic clk);
  import alu_pkg::*;

  // ---- signals ----
  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  // ---- timing for the driver ----
  clocking drv_cb @(posedge clk);
    default input #1step output #1;
    output valid_in, a, b, op;
    input  valid_out, result;
  endclocking

  // ---- timing for the monitor ----
  clocking mon_cb @(posedge clk);
    default input #1step;
    input valid_in, a, b, op, valid_out, result;
  endclocking

  // ---- views ----
  modport DUT (input clk, rst_n, valid_in, a, b, op, output valid_out, result);
  modport DRV (clocking drv_cb, input clk, rst_n);
  modport MON (clocking mon_cb, input clk, rst_n);
endinterface
```

Line by line

Line	Explanation
`interface alu_if (input logic clk);`	The clock comes from outside, so that the DUT and the interface share one clock.
`import alu_pkg::*;`	Makes the opcode type `alu_op_e` visible inside the interface.
`logic rst_n;` ... `logic [8:0] result;`	One signal per DUT port (except the clock), with the same names and types as the ports. `rst_n` is driven by `tb_top`.
`clocking drv_cb @(posedge clk);`	The driver's clocking block, synchronous to the rising edge.
`default input #1step output #1;`	Inputs are sampled just before the edge; outputs change one time unit after it.
`output valid_in, a, b, op;`	The driver drives every DUT input except reset.
`input valid_out, result;`	The driver may also look at the DUT outputs, for protocols where the driver must react to the DUT. The ALU driver does not need them, but they are available.

Line	Explanation
`clocking mon_cb @(posedge clk);`	The monitor's clocking block.
`input valid_in, a, b, op, valid_out, result;`	Everything is an input: the monitor observes both what went into the DUT and what came out, and can drive nothing.
`modport DUT (...)`	The design's view: plain signals with directions. It has no clocking block, because RTL does not use one.
`modport DRV (clocking drv_cb, input clk, rst_n);`	The driver's view: the clocking block, plus `clk` and `rst_n` as plain inputs. The raw signals `a`, `b` and so on are **not** listed, so the driver can reach them only through `drv_cb`.
`modport MON (clocking mon_cb, input clk, rst_n);`	The monitor's view.

Why `rst_n` is not inside the clocking blocks. The reset is asynchronous: it is applied while the clock may not even be running, and the DUT reacts to it immediately. A clocking block only acts at clock edges. So reset is driven directly by `tb_top` and is visible to the driver and the monitor as a plain input, for example to wait with `@(posedge vif.rst_n)` or `wait (vif.rst_n);`.

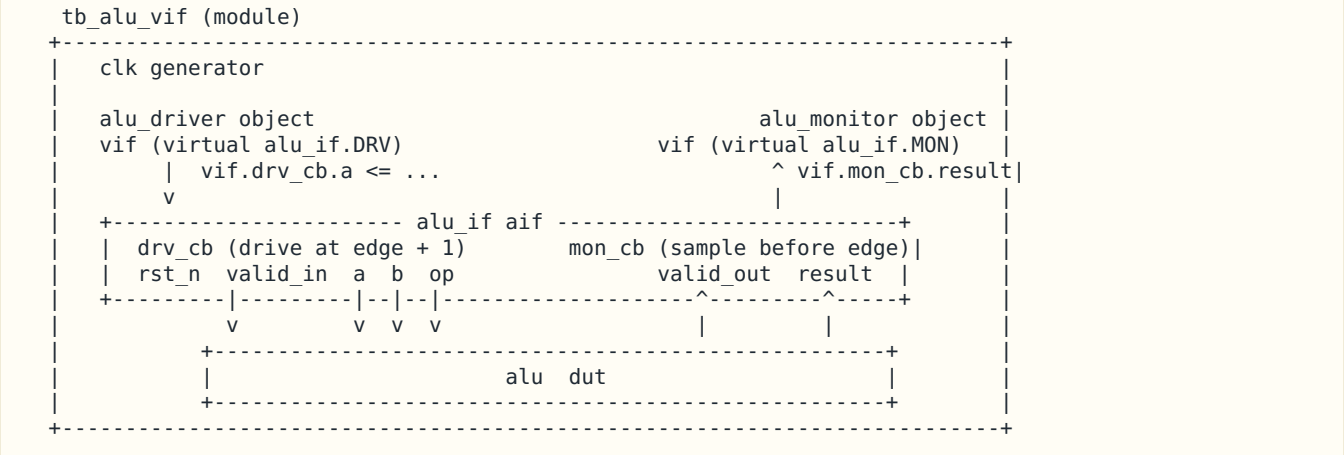
Why the driver and the monitor have separate clocking blocks. Their directions differ. The DUT inputs are outputs of the driver and inputs of the monitor. One clocking block cannot give a signal two directions.

How the three users see the interface

Signal	`modport DUT`	`modport DRV`	`modport MON`
`clk`	input	input	input
`rst_n`	input	input	input
`valid_in`, `a`, `b`, `op`	input	`drv_cb` output (synchronous drive)	`mon_cb` input (sampled)
`valid_out`, `result`	output	`drv_cb` input (sampled)	`mon_cb` input (sampled)

5.2 Complete Example: Classes Driving and Monitoring the ALU

What is built



The driver and the monitor here are simplified: the driver's `drive` task takes the operand values as arguments, and the monitor prints what it sees. Module 9 gives them mailboxes and transaction objects; the pin-level code stays as it is here.

The classes

alu_vif_classes.sv

```
import alu_pkg::*;

class alu_driver;

  virtual alu_if.DRV vif;

  function new(virtual alu_if.DRV vif);
    this.vif = vif;
  endfunction

  // Drive one operation: one cycle with valid_in = 1, then valid_in = 0
  task drive(bit [7:0] a, bit [7:0] b, alu_op_e op);
    @(vif.drv_cb);
    vif.drv_cb.valid_in <= 1'b1;
    vif.drv_cb.a        <= a;
    vif.drv_cb.b        <= b;
    vif.drv_cb.op       <= op;
    @(vif.drv_cb);
    vif.drv_cb.valid_in <= 1'b0;
    $display("[%0t] DRV: %s a=%0d b=%0d accepted by the DUT", $time, op.name(), a, b);
  endtask

endclass

class alu_monitor;

  virtual alu_if.MON vif;

  function new(virtual alu_if.MON vif);
    this.vif = vif;
  endfunction

  task run();
    logic [7:0] a, b;        // the most recent request seen on the DUT inputs
    alu_op_e    op;
    forever begin
      @(vif.mon_cb);
      if (vif.mon_cb.valid_out)
        $display("[%0t] MON: %s a=%0d b=%0d -> result=%0d",
                 $time, op.name(), a, b, vif.mon_cb.result);
      if (vif.mon_cb.valid_in) begin
        a  = vif.mon_cb.a;
        b  = vif.mon_cb.b;
        op = vif.mon_cb.op;
      end
    end
  endtask

endclass
```

The top module

Compile in this order: `alu_pkg.sv` and `alu.sv` (from `examples/module-01/`), `alu_if.sv`, `alu_vif_classes.sv`, then this file.

tb_alu_vif.sv

```
module tb_alu_vif;
  import alu_pkg::*;
```

```
  logic clk = 0;
  always #5 clk = ~clk;

  alu_if aif (clk);                          // physical interface

  alu dut (                                  // DUT with ordinary ports
    .clk       (clk),
    .rst_n     (aif.rst_n),
    .valid_in  (aif.valid_in),
    .a         (aif.a),
    .b         (aif.b),
    .op        (aif.op),
    .valid_out (aif.valid_out),
    .result    (aif.result)
  );

  alu_driver  drv;
  alu_monitor mon;

  initial begin
    // Reset is asynchronous, so the reset-time values are assigned directly
    aif.rst_n    = 0;
    aif.valid_in = 0;
    aif.a        = '0;
    aif.b        = '0;
    aif.op       = ALU_ADD;

    drv = new(aif);                          // physical -> virtual (modport DRV)
    mon = new(aif);                          // physical -> virtual (modport MON)

    repeat (2) @(negedge clk);
    aif.rst_n = 1;                           // released at time 20, away from the edge

    fork
      mon.run();                             // the monitor runs in the background
    join_none

    drv.drive(8'd200, 8'd100, ALU_ADD);
    drv.drive(8'd5,   8'd10,  ALU_SUB);
    drv.drive(8'hF0,  8'h3C,  ALU_XOR);

    repeat (3) @(posedge clk);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[35] DRV: ALU_ADD a=200 b=100 accepted by the DUT
[45] MON: ALU_ADD a=200 b=100 -> result=300
[55] DRV: ALU_SUB a=5 b=10 accepted by the DUT
[65] MON: ALU_SUB a=5 b=10 -> result=507
[75] DRV: ALU_XOR a=240 b=60 accepted by the DUT
[85] MON: ALU_XOR a=240 b=60 -> result=204
```

Line by line: the driver

Line	Explanation
`import alu_pkg::*;`	At the top of the file, so that the classes can use `alu_op_e`.
`virtual alu_if.DRV vif;`	The handle, restricted to the driver's modport. Through it the class can use `vif.drv_cb`, `vif.clk` and `vif.rst_n`, and nothing else.
`function new(virtual alu_if.DRV vif);`	The handle arrives through the constructor.
`@(vif.drv_cb);`	Wait for the next rising clock edge, as defined by the clocking block.

Line	Explanation
`vif.drv_cb.valid_in <= 1'b1;` and the three lines after it	Synchronous drives. All four signals change together, one time unit after this edge. The DUT, which sampled at the edge itself, has not seen them yet.
`@(vif.drv_cb);`	Wait one cycle. At this edge the DUT samples `valid_in` = 1 together with `a`, `b` and `op`.
`vif.drv_cb.valid_in <= 1'b0;`	End of the request: `valid_in` goes low one time unit after this edge.
`$display(... op.name() ...)`	Printed at the edge at which the DUT captured the request.

Line by line: the monitor

Line	Explanation
`virtual alu_if.MON vif;`	Restricted to the monitor's modport: only sampled inputs. An attempt to drive anything from this class does not compile.
`logic [7:0] a, b;`	Four-state, so that an X on the DUT inputs would be preserved (Module 1, Section 2.1).
`forever begin @(vif.mon_cb);`	The monitor looks at the bus at every clock edge for the whole simulation.
`if (vif.mon_cb.valid_out)`	The sampled value of `valid_out`. When it is 1, `result` belongs to the request that was captured one cycle earlier, which is held in `a`, `b` and `op`.
`if (vif.mon_cb.valid_in) begin ... end`	A request was presented to the DUT at this edge. Its operands are remembered for the next cycle. This check comes after the result check so that a result is always paired with the previous request, even if requests arrive in consecutive cycles.

Line by line: the top module

Line	Explanation
`alu_if aif (clk);`	The one physical interface instance.
`alu dut (... .a(aif.a) ...);`	The DUT's ordinary ports are connected to the interface signals (Section 1.5).
`aif.rst_n = 0;` ... `aif.op = ALU_ADD;`	During reset the top module assigns the signals directly, by instance name. A clocking block drive could not do this at time 0, because it only acts at a clock edge. A known `op` also keeps the `unique case` in the ALU from reporting an X opcode.
`drv = new(aif);`	The instance is passed to a constructor that expects `virtual alu_if.DRV`. The handle inside the driver is restricted to that modport. Writing `new(aif.DRV)` is equivalent.
`repeat (2) @(negedge clk); aif.rst_n = 1;`	Reset is held for two falling edges and released at time 20.
`fork mon.run(); join_none`	The monitor's endless loop is started as a parallel thread (Module 7).
`drv.drive(...)` three times	Hand-written stimulus. Module 9 replaces these calls with a generator and a mailbox.
`repeat (3) @(posedge clk); $finish;`	Leaves time for the last result to be observed.

Execution flow of the first operation

Time	DUT	Driver	Monitor
20	Reset released	`drive()` is called and waits for `drv_cb`	Started, waits for `mon_cb`
25	Samples `valid_in` = 0	Wakes; requests `valid_in` = 1, `a` = 200, `b` = 100, `op` = `ALU_ADD`	Samples `valid_in` = 0, `valid_out` = 0: nothing to do
26	Inputs change		
35	Samples `valid_in` = 1: stores `result` = 300, sets `valid_out` = 1	Wakes; requests `valid_in` = 0; prints the `DRV` line; `drive()` returns	Samples `valid_in` = 1 (value before the edge): remembers 200, 100, `ALU_ADD`. `valid_out` is still 0 in the sample
36	`valid_in` goes low		
45	Samples `valid_in` = 0: sets `valid_out` = 0	Second `drive()` wakes and drives the next request	Samples `valid_out` = 1 and `result` = 300: prints the `MON` line

The second and third operations repeat this pattern twenty time units later each. Check the results: 5 - 10 in 9 bits is `1_1111_1011` = 507 (bit 8 is the borrow), and `F0 ^ 3C` = `CC` = 204.

Each number in the log is determined by the clocking blocks, not by the order in which the simulator runs the driver, the monitor and the DUT at an edge. That is the property a testbench must have.

> **Common mistake:** Driving DUT inputs both through the clocking block and by direct assignment during normal operation. The direct assignments at time 0 above are acceptable because they happen once, before any synchronous drive. After that, use the clocking block only. And never drive a clocking block output with a continuous `assign` as well: that is illegal.

> **Verification application:** This is the complete "signal level" of a layered testbench. In Module 9 the generator creates `alu_txn` objects, a mailbox carries them to `alu_driver`, whose `run()` task calls exactly this `drive` logic; `alu_monitor` builds an `alu_txn` from what it samples and sends it through another mailbox to `alu_scoreboard`.

Interview questions

Q1. Explain how a driver class drives the DUT. The driver has a virtual interface handle that the top module set to the physical interface connected to the DUT. For each transaction the driver waits for the clocking block event, `@(vif.drv_cb)`, and applies the transaction's fields with synchronous drives, `vif.drv_cb.sig <= value`. The clocking block puts the values on the interface signals just after the clock edge, and the DUT samples them at the next edge.

Q2. What is the difference between how the driver and the monitor use the interface? The driver drives DUT inputs through a clocking block whose signals are outputs. The monitor only samples, through a clocking block (and modport) in which every signal is an input; it never drives.

Q3. Why is reset not driven through the clocking block? Reset is asynchronous and must be applied regardless of the clock, possibly before the clock runs. Clocking block drives take effect only at clock events.

5.3 Bidirectional Signals in an Interface

The problem

Some pins are driven by the DUT at certain times and by the testbench at others: a memory data bus, I2C `sda`, a GPIO pin. Three facts from earlier collide here:

1. A signal with more than one driver must be a **net** (`wire`), because only a net resolves several drivers (Module 1, Section 2.4).
2. A net cannot be assigned procedurally.
3. A class can only make procedural assignments through its virtual interface.

So a class cannot drive a `wire` in the interface directly.

The pattern

Put three things in the interface: the `wire` itself, a `logic` variable holding the value the testbench wants to drive, and a `logic` output-enable. A continuous assignment in the interface is the testbench's tri-state driver. The class writes only the two variables.

```
   class (procedural)             interface                        DUT
   ------------------             ---------                       -----
   vif.data_drv = 8'hA5; ---->  data_drv ---+
                                            |  assign data =
   vif.data_oe  = 1;     ---->  data_oe ----+   data_oe ? data_drv : 'z;
                                            |
                                            v
   value = vif.data;     <----  wire data <==========================> inout data
                                 (resolved value of both drivers)
```

Complete example

tb_bidir.sv

```
interface bidir_if (input logic clk);
  logic       wr;              // testbench -> DUT: store the value that is on the bus
  logic       rd;              // testbench -> DUT: put the stored value on the bus
  wire  [7:0] data;            // shared bus: a net, because it has two drivers

  logic [7:0] data_drv;        // value the testbench wants on the bus
  logic       data_oe;         // 1 = the testbench is driving the bus

  assign data = data_oe ? data_drv : 'z;       // the testbench's tri-state driver
endinterface

module bidir_reg (input logic clk, input logic wr, input logic rd, inout wire [7:0] data);
  logic [7:0] store;
  always_ff @(posedge clk) if (wr) store <= data;
  assign data = rd ? store : 'z;               // the DUT's tri-state driver
endmodule

class bidir_driver;

  virtual bidir_if vif;

  function new(virtual bidir_if vif);
    this.vif = vif;
  endfunction

  task write(bit [7:0] value);
    @(negedge vif.clk);
    vif.data_drv = value;
```

```
    vif.data_oe  = 1;          // take the bus
    vif.wr       = 1;
    @(negedge vif.clk);
    vif.wr       = 0;
    vif.data_oe  = 0;          // release the bus
  endtask

  task read(output logic [7:0] value);
    @(negedge vif.clk);
    vif.rd = 1;                // the DUT drives the bus now
    @(negedge vif.clk);
    value  = vif.data;         // reading a net through the handle is allowed
    vif.rd = 0;
  endtask

endclass

module tb_bidir;

  logic clk = 0;
  always #5 clk = ~clk;

  bidir_if     bif (clk);
  bidir_reg    dut (.clk(clk), .wr(bif.wr), .rd(bif.rd), .data(bif.data));
  bidir_driver drv;
  logic [7:0]  value;

  initial begin
    bif.wr = 0;  bif.rd = 0;  bif.data_oe = 0;  bif.data_drv = '0;
    drv = new(bif);

    #1 $display("[%0t] bus idle       : data = %h", $time, bif.data);
    drv.write(8'hA5);
    #1 $display("[%0t] after the write: data = %h", $time, bif.data);
    drv.read(value);
    $display("[%0t] value read back: %h", $time, value);
    #1 $display("[%0t] bus idle again : data = %h", $time, bif.data);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[1] bus idle       : data = zz
[21] after the write: data = zz
[40] value read back: a5
[41] bus idle again : data = zz
```

Line by line

Line	Explanation
`wire [7:0] data;`	A net. It will have two continuous drivers: one in the interface, one in the DUT.
`logic [7:0] data_drv; logic data_oe;`	Variables that the class can assign procedurally.
`assign data = data_oe ? data_drv : 'z;`	When `data_oe` is 1 the testbench's value goes onto the bus; otherwise this driver is high impedance and does not disturb the bus.
`inout wire [7:0] data`	The DUT's bidirectional port. `inout` ports must be nets.
`assign data = rd ? store : 'z;`	The DUT drives the bus only while `rd` is 1.
`vif.data_oe = 1;` ... `vif.data_oe = 0;`	The driver takes the bus for one cycle and then releases it. Forgetting to release it would leave two drivers fighting during a later read.

Line	Explanation
`value = vif.data;`	The resolved value of the net: here only the DUT is driving, so it is the DUT's value.
`// vif.data = 8'h55;` (not in the code)	Would be a compile error: a net cannot be assigned procedurally.

Execution flow

Time	Event
1	Neither side drives: `data` = `zz`.
10	`write` drives `A5` onto the bus and sets `wr`.
15	Rising edge: the DUT stores `A5`.
20	`write` clears `wr` and releases the bus.
21	Printed: the bus is floating again, `zz`.
30	`read` sets `rd`. The DUT drives `A5`.
40	`read` copies `A5` from the bus and clears `rd`. Printed.
41	The DUT has stopped driving: `zz`.

> **Interview trap:** "A class needs to drive an `inout` pin of the DUT. How?" Not directly: the pin must be a `wire`, and a class cannot assign a net. The interface holds a `logic` drive value and a `logic` output-enable, plus `assign pin = oe ? drive_value : 'z;`. The class assigns the two variables and reads the wire.

> **Common mistake:** Both sides driving at once. If the testbench keeps `data_oe` at 1 while the DUT also drives, any bit where the two values differ becomes X. An X on a bidirectional bus during a read almost always means the testbench did not release the bus.

A clocking block can also contain such a signal as an `inout`: the clocking block then samples the net like an input and drives it like an output, and driving it with `'z` releases it. The explicit pattern above is easier to read and to debug, and is the form most often asked about.

Interview questions

Q1. Why must a bidirectional signal in an interface be a `wire`? It has more than one driver, the DUT and the testbench. Only a net resolves multiple drivers; a `logic` variable with two continuous drivers is an error.

Q2. How does a class-based driver drive a `wire` in the interface? Through an intermediate variable and an output-enable in the interface, with a continuous tri-state assignment from them to the wire. The class writes the variables; the assignment drives the net.

Q3. What does X on a bidirectional bus usually indicate? Contention: two drivers are active with different values at the same time.

5.4 Program Blocks

This section is for awareness. You will be asked what a program block is, and why most current testbenches do not use one.

Definition

A `program` block (`program` ... `endprogram`) is a container for testbench code. It looks like a module but is executed differently: all code in a program runs in the **Reactive** region of the time step, after the design's Active and NBA regions are finished. The testbench therefore always sees the design's settled values for the current time step, and its own drives reach the design after the design has sampled. It was introduced to separate testbench and design scheduling.

```
program automatic alu_test_prog (alu_if aif);

  initial begin
    @(aif.drv_cb);
    aif.drv_cb.valid_in <= 1'b1;
    // ...
  end

endprogram

module tb_top;
  logic clk = 0;
  always #5 clk = ~clk;
  alu_if        aif  (clk);
  // alu        dut  (...);
  alu_test_prog test (aif);        // a program is instantiated like a module
endmodule
```

Rules

A program block may contain	A program block may not contain
`initial` blocks	`always` blocks
Variables, classes, tasks, functions	Instances of modules or interfaces
Clocking blocks	User-defined primitives
Assertions, covergroups	

Other points:

- When every `initial` block of every program has finished, the simulation ends automatically, as if `$finish` had been called. `$exit` ends a program explicitly.
- A program can call tasks and functions in modules. A module cannot call tasks or functions of a program.

Module, program and the practice today

Property	`module`	`program`
Intended for	Design and testbench structure	Testbench code only
Region where its code runs	Active	Reactive
`always` blocks	Yes	No

Property	`module`	`program`
Can instantiate modules and interfaces	Yes	No
Ends the simulation when its `initial` blocks finish	No	Yes
Race with the DUT at a clock edge	Possible, unless clocking blocks or nonblocking drives are used	Avoided by the region separation

In current practice, and in UVM, the testbench is a set of classes in a package, started from an `initial` block in a top-level **module**, and race freedom comes from clocking blocks in the interface. Program blocks are therefore rare in new code: they add restrictions (no `always`, no instances), they do not help code that lives in classes called from a module, and a mixture of program-based and module-based testbench code can itself create ordering surprises. You still need to recognise them in older testbenches.

Interview questions

Q1. What is a program block? A testbench container whose code executes in the Reactive region, after the design has finished evaluating for the time step. It was introduced to avoid races between the testbench and the DUT.

Q2. What is the difference between a module and a program? A program cannot contain `always` blocks or instantiate modules and interfaces; its code runs in the Reactive region instead of the Active region; and the simulation ends when all program `initial` blocks have completed.

Q3. Are program blocks used in UVM testbenches? Generally not. UVM testbenches are class-based, started from a module, and rely on clocking blocks (and nonblocking drives) for race avoidance.

5.5 Interface, Modport, Clocking Block and Virtual Interface Compared

These four are often confused. Each answers a different question.

Construct	Question it answers	Static or dynamic	Used by
`interface`	**Which** signals belong to this bus?	Static instance	DUT connection and testbench
`modport`	**Who** may read and write each signal (direction)?	Declaration inside the interface	Module ports and virtual interface declarations
`clocking` block	**When** are signals sampled and driven (timing)?	Declaration inside the interface	Testbench code only
`virtual interface`	**How** does a class reach the interface instance?	Dynamic handle	Classes

Modport versus clocking block

Property	Modport	Clocking block
Purpose	Direction and visibility	Timing (synchronisation to a clock)
Specifies a clock	No	Yes: the clocking event

Property	Modport	Clocking block
Specifies skews	No	Yes
Direction is from the point of view of	Whoever uses the modport	The testbench
Used by the DUT	Yes	No
Synthesizable	Yes	No
Prevents races	No	Yes
Prevents a wrong-direction write	Yes	Yes, for the signals it lists
How a signal is accessed	`vif.sig`	`vif.cb.sig`
Can be combined	A modport can contain a clocking block: `modport DRV (clocking drv_cb, ...);`	

The complete picture

```
1. interface alu_if        declares signals, drv_cb, mon_cb, modports       (static)
2. tb_top                  alu_if aif (clk);  alu dut (... aif.a ...);       (static)
3. class alu_driver        virtual alu_if.DRV vif;                           (dynamic)
4. tb_top, at run time     drv = new(aif);            physical -> virtual
5. class methods           @(vif.drv_cb);  vif.drv_cb.a <= value;           (race-free)
```

Part 6: Module 8 Revision and Practice

6.1 Revision Summary

The four constructs on one page

Construct	What it is	Declared	Key syntax
Interface	A named bundle of signals, instantiated once and shared	Outside modules, like a module	`interface alu_if (input logic clk); ... endinterface`
Modport	A named view: which signals, which direction, for one role	Inside the interface	`modport DUT (input a, output result);`
Clocking block	Sampling and driving times relative to a clock	Inside the interface	`clocking cb @(posedge clk); default input #1step output #1; ... endclocking`
Virtual interface	A handle that lets a class reach an interface instance	In a class	`virtual alu_if.DRV vif;`

Rules to remember

Interfaces

- An interface instance holds the signals once. Interface ports and virtual interfaces are references to it, never copies.
- It may contain signals, parameters, ports, modports, clocking blocks, tasks, functions, `initial` / `always` / `assign` and assertions. It may not contain module instances.
- The clock is normally an input port of the interface, generated once in the top module.
- A DUT with ordinary ports is connected signal by signal, once, in the top module: `.a(aif.a)`.
- `logic` signals in an interface follow the single-driver rule. A signal with two drivers must be a `wire`.

Modports

- Directions are from the user's side. What the testbench drives is an `output` in the testbench's modport.
- A modport is selected in the module header (`alu_if.DUT p`), at the instance (`aif.DUT`), or in a virtual interface declaration (`virtual alu_if.DRV vif`).
- Writing an `input`, or touching an unlisted signal, through a modport is a compile error.
- Modports do not stop hierarchical access, add no hardware and define no timing.
- A modport can contain a clocking block and can import tasks.

Interface arrays

- `my_if bus [4] (clk);` creates four independent instances that share the port connections.
- The index must be a constant. For variable indexing, use an array of virtual interfaces.

Virtual interfaces

- A class cannot instantiate an interface and has no ports. It holds a `virtual <interface> vif` handle instead.
- The handle is `null` until the top module assigns the physical instance to it, usually through the constructor.

- Using a null handle is a fatal run-time error.
- A handle for a parameterised interface must repeat the same parameter values.
- Nets cannot be assigned through a virtual interface; only variables can.

Clocking blocks

- Directions are from the testbench's side: `input` = sampled, `output` = driven.
- Default skews: input `#1step`, output `#0`.
- `#1step` samples in the Preponed region (value before the edge). `#0` on an input samples in the Observed region (value after the edge).
- Wait with `@(vif.cb)`, read `vif.cb.sig`, drive with `vif.cb.sig <= value`. Only `<=` is allowed; an output-only clockvar cannot be read.
- A drive executed away from a clocking event takes effect as if executed at the next one.
- `##N` needs a default clocking and is not available in classes; use `repeat (N) @(vif.cb)`.
- Reset is asynchronous and stays outside the clocking block.

Scheduler regions (simplified order)

Preponed (sample) → Active (design reads, blocking assignments) → Inactive (`#0`) → NBA (nonblocking updates) → Observed (assertions, `#0` input sampling, clocking event) → Reactive (program code) → Re-NBA (zero-skew clocking drives) → Postponed (`$monitor`, `$strobe`).

The recipe for connecting a class to a DUT

Step	Code
1. Interface with clocking blocks and modports	`interface alu_if (input logic clk); ... endinterface`
2. Instantiate in the top module and connect the DUT	`alu_if aif (clk); alu dut (.a(aif.a), ...);`
3. Handle in the class	`virtual alu_if.DRV vif;`
4. Constructor stores the handle	`function new(virtual alu_if.DRV vif); this.vif = vif; endfunction`
5. Top module passes the instance	`drv = new(aif);`
6. Class drives and samples synchronously	`@(vif.drv_cb); vif.drv_cb.a <= 8'd5;`

Synthesizable or simulation-only

Synthesizable	Simulation-only
Interface with signals and parameters	Clocking blocks
Modports	Virtual interfaces
Interface ports of modules	Program blocks
Synthesizable `always` / `assign` logic and functions inside an interface	`initial` blocks, delays, clock generators, testbench tasks and assertions inside an interface

6.2 Interview Questions and Answers

The questions inside Parts 1 to 5 cover the direct definitions. The questions below are additional.

Basic level

Q1. What can an interface contain? Signals (nets and variables), ports, parameters, modports, clocking blocks, tasks, functions, `initial` and `always` blocks, continuous assignments, assertions, covergroups and instances of other interfaces. It cannot contain module instances.

Q2. How is an interface different from a module? A module describes functionality and can instantiate modules. An interface describes the connection between blocks, cannot instantiate modules, can be passed as a single port, supports modports, and can be referenced from classes through a virtual interface.

Q3. Write an interface for a simple bus with `valid`, `ready` and 8-bit `data`. `interface bus_if (input logic clk); logic valid; logic ready; logic [7:0] data; endinterface`

Q4. How do you instantiate an interface that has no ports? With empty parentheses: `bus_if bus ();`.

Q5. What directions can a modport give a signal? `input`, `output`, `inout` and `ref`.

Q6. What is the default value of a virtual interface? `null`.

Q7. What are the default skews of a clocking block? `#1step` for inputs and `#0` for outputs.

Q8. Which operator drives a clocking block output? The nonblocking-style operator `<=`. A blocking assignment to a clockvar is illegal.

Q9. Write the declaration of a virtual interface for `alu_if` restricted to the monitor modport. `virtual alu_if.MON vif;`

Q10. Where is the physical interface instantiated? In the top-level testbench module, next to the DUT.

Q11. Can a class have a port? No. A class receives everything through constructor or method arguments, or by assignment to its properties.

Q12. What does `@(vif.mon_cb);` do? It waits for the next clocking event of `mon_cb` (the next rising clock edge). When the process resumes, `vif.mon_cb.<signal>` holds the values sampled at that edge.

Intermediate level

Q13. Why is the monitor given a clocking block with only inputs? The monitor must observe without disturbing. With only inputs it cannot drive any signal, and it samples every signal at a race-free point, just before the clock edge.

Q14. Why do the driver and the monitor use different clocking blocks? The DUT's inputs are outputs for the driver and inputs for the monitor. A signal has one direction inside a clocking block, so two blocks are needed.

Q15. A signal is driven through a clocking block at the clock edge at time 100 with output skew `#2`. When does the DUT see it? The signal changes at time 102. The DUT, which samples at rising edges, sees it at the next rising edge after 102.

Q16. The DUT changes an output at the clock edge at time 100. When does `vif.mon_cb.out` show the new value? At the next clocking event. At time 100 the monitor's sample was taken in the Preponed region, before the change, so it still shows the old value.

Q17. What is the difference between `vif.a` and `vif.drv_cb.a`? `vif.a` is the raw interface signal: reading gives its current value, writing changes it at once, and both can race with the DUT. `vif.drv_cb.a` is the clockvar: writes are applied at the clock edge plus the output skew, and reads (for an input) return the value sampled before the edge.

Q18. Can the same interface signal be in two clocking blocks? Yes. In `alu_if`, `valid_in`, `a`, `b` and `op` are outputs of `drv_cb` and inputs of `mon_cb`.

Q19. How can a clocking block be triggered by the falling edge, or by both edges? By its clocking event: `clocking cb @(negedge clk);` or `clocking cb @(clk);` for both edges.

Q20. What is the difference between `##2` and `repeat (2) @(posedge clk)`? In a scope with a default clocking, `##2` waits two clocking events of that block, which is the same amount of time. `##2` is tied to the default clocking block and its event; it cannot be used in a class. `repeat (2) @(posedge clk)` waits on the raw clock signal.

Q21. How do you pass an interface to a task? As a virtual interface argument: `task automatic send(virtual bus_if vif, input logic [7:0] d);`. The caller passes the interface instance or a handle.

Q22. Two driver objects are created and both are given the same interface instance. What happens? Both handles refer to the same signals, so both objects drive the same pins. With procedural or clocking block drives, the last write wins and the stimulus is corrupted. Each interface instance should have one driver.

Q23. Can an interface be passed to a class without a virtual interface, for example by `ref` argument? No. Individual signals can be passed by `ref` to an automatic task, but an interface instance is not a variable. The only handle to an interface instance is a virtual interface.

Q24. Why might a design team avoid interfaces in RTL port lists while the verification team uses them heavily? Some synthesis, lint and physical design flows handle interface ports less uniformly than plain ports, and the synthesised netlist gets tool-generated port names. Plain RTL ports avoid that. In the testbench there are no such concerns, and interfaces are required for class-based code.

Q25. What happens to modport checking when the testbench accesses `tb_top.aif.a` directly? Nothing is checked. Modports apply only to access through a port or a virtual interface that selected the modport.

Advanced and scenario-based

Q26. A test passes on one simulator and fails on another with results shifted by one clock cycle. What do you suspect? A race between the testbench and the DUT. Typical causes: the driver assigns DUT inputs with blocking assignments at the active clock edge; the monitor reads raw signals at the clock edge; the code waits on `@(posedge vif.clk)` but reads clocking block samples. The fix is to drive and sample only through clocking blocks and to wait on the clocking block event.

Q27. The monitor reports every result one cycle earlier than expected and the scoreboard pairs it with the wrong request. The monitor uses a clocking block. What is wrong? Probably the input skew is `#0` instead of `#1step`. With `#0` the monitor samples in the Observed region, after the DUT's registers have updated, so it sees the output of this edge instead of the value before it.

Q28. The simulation dies at time 0 with a null handle error in the driver's `run` task. List the possible causes. The driver object itself was never constructed; or its virtual interface was never assigned: not passed to the constructor, the constructor wrote `vif = vif` without `this.`, the environment did not forward the handle, or `run()` was started before the connection was made.

Q29. A driver uses `vif.drv_cb.rst_n <= 0` to apply reset at the start of the test, and the DUT is never reset. Why? Clocking block drives only happen at clocking events. If the clock is not running yet, or the first edge comes later than expected, the reset is not applied when needed. Reset is asynchronous and should be driven directly (from the top module or by direct assignment), not through a clocking block.

Q30. You need to verify a design with eight identical ports. How do you organise the interfaces and the testbench? Declare an array of eight interface instances and connect element `i` to DUT port `i` in a generate loop. Copy the instances into an array of eight virtual interfaces. Create eight agents in a loop and give agent `i` the handle `vifs[i]`. The driver and monitor classes are written once.

Q31. Explain what happens, region by region, when a driver executes `vif.drv_cb.a <= 5` after `@(vif.drv_cb)` with `output #0`. At the clock edge, the DUT's flip-flops sample their inputs in the Active region and update in the NBA region. The clocking block event is triggered in the Observed region and the driver process resumes. Its drive is applied to the raw signal `a` in the Re-NBA region of the same time step. The DUT has already sampled, so it sees the value 5 at the next clock edge.

Q32. Can you drive the same DUT input from a clocking block and from a continuous assignment? No. A variable associated with a clocking block output may not also be driven by a continuous assignment. Choose one mechanism.

Q33. What is a BFM and how does it relate to interfaces? A bus functional model is code that performs bus-level operations (read, write) by toggling the pins with the correct protocol timing. It can be implemented as tasks inside the interface, called from tests or classes, or as a driver class that uses a virtual interface. The first form is preferred when the model must run in an emulator; the second is the normal class-based style.

Q34. In UVM, how does the driver get its virtual interface? The top module stores the interface instance in the configuration database with `uvm_config_db #(virtual alu_if)::set(...)`, and the driver (or its agent) retrieves it in the build phase with `uvm_config_db #(virtual alu_if)::get(...)`, reporting a fatal error if it is missing.

Q35. Why is `#1step` better than a fixed input skew such as `#1`? `#1step` always means "the last value before the clock event", independent of the time unit, the clock period and the time precision. A fixed number is measured in time units and may be wrong for a different timescale or a faster clock; it is used only when a specific setup time must be modelled, as in gate-level simulation.

Q36. What would you put in the interface besides signals, in a real project? Clocking blocks for driver and monitor, modports for DUT, driver and monitor, protocol assertions, sometimes functional coverage of the bus, and occasionally helper tasks such as `wait_clocks(n)` or a reset task.

6.3 Output-Prediction Questions

Work out the answer on paper first. P1, P4 and P5 need `counter_dut.sv` or `dff.sv` from this module to be compiled first.

P1.

```
module p1;
  logic clk = 0;
  always #5 clk = ~clk;
  cnt_if  cif (clk);
  counter dut (cif);
  virtual cnt_if v1, v2;
  initial begin
    v1 = cif;
    v2 = v1;
    v1.rst_n = 0;
    v2.en    = 1;
    @(negedge clk);
    v2.rst_n = 1;
    repeat (2) @(negedge clk);
    $display("%0d %0d %b", v1.count, cif.count, v1 == v2);
    v2 = null;
    $display("%b %b", v1 == null, v2 == null);
    $finish;
  end
endmodule
```

Answer: `2 2 1` then `0 1`. Both handles refer to the one instance `cif`, so a write through either reaches the same signals and the comparison `v1 == v2` is true. Reset is released at time 10 and two rising edges (15 and 25) follow before time 30. Setting `v2` to `null` changes only that handle.

P2.

```
module p2;
  logic       clk = 0;
  logic [3:0] n   = 0;
  always #5 clk = ~clk;
  always @(posedge clk) n <= n + 4'd2;
  clocking cb @(posedge clk);
    input n;
  endclocking
  initial begin
    repeat (3) begin
      @(cb);
      $display("%0d", cb.n);
    end
    $finish;
  end
endmodule
```

Answer: `0`, `2`, `4` on three lines. The default input skew is `#1step`, so at each rising edge the clocking block holds the value of `n` from just before that edge: 0 at time 5, 2 at time 15, 4 at time 25.

P3.

```
module p3;
  logic       clk = 0;
  logic [7:0] d   = 8'd1;
  always #5 clk = ~clk;
  clocking cb @(posedge clk);
    output #3 d;
  endclocking
  always @(d) $display("[%0t] d = %0d", $time, d);
  initial begin
    @(cb);
    cb.d <= 8'd2;
    @(negedge clk);
    cb.d <= 8'd3;
    repeat (2) @(cb);
    $finish;
  end
endmodule
```

Answer: `[8] d = 2` then `[18] d = 3`. The first drive is executed at the clocking event of time 5 and is applied 3 time units later. The second is executed at time 10, which is not a clocking event, so it is treated as if executed at the next event (15) and applied at 18.

P4.

```
module p4;
  logic clk = 0, d = 0, q;
  always #5 clk = ~clk;
  dff dut (.*);
  initial begin
    @(posedge clk);
    d <= 1;
    @(posedge clk);
    $display("%b", q);
    @(negedge clk);
    $display("%b", q);
    $finish;
  end
endmodule
```

Answer: `0` then `1`, on every simulator. Compare with `tb_race.sv`: the only change is `d <= 1` instead of `d = 1`. The nonblocking update of `d` happens in the NBA region of time 5, after the flip-flop has sampled `d` = 0. At time 15 the flip-flop samples 1, but `q` changes in the NBA region, after the `$display` in the Active region has printed the old value 0. At time 20, `q` is 1.

P5.

```
module p5;
  logic clk = 0;
  always #5 clk = ~clk;
  cnt_if cif [3] (clk);
  for (genvar i = 0; i < 3; i++) begin : g
    counter dut (cif[i]);
  end
  virtual cnt_if v [3];
  initial begin
    v[0] = cif[0];  v[1] = cif[1];  v[2] = cif[2];
    foreach (v[i]) begin
      v[i].rst_n = 0;
      v[i].en    = (i != 1);
    end
    @(negedge clk);
    foreach (v[i]) v[i].rst_n = 1;
    repeat (3) @(negedge clk);
    v[2] = v[1];
    $display("%0d %0d %0d", v[0].count, v[1].count, v[2].count);
    $finish;
  end
endmodule
```

Answer: `3 0 0`. Counters 0 and 2 are enabled and count three edges (15, 25, 35); counter 1 is disabled. Before printing, handle `v[2]` is re-pointed to the instance that `v[1]` refers to, so the third value printed is counter 1's value, 0. Counter 2 still holds 3, but no handle refers to it any more.

P6. `tb_race.sv` of Section 4.1 prints `q` once. What are the possible outputs, and why?

Answer: `q = 0` or `q = 1`, depending on the simulator. The testbench changes `d` with a blocking assignment in the same time step and region in which the flip-flop samples it, so the order of the two processes decides the result.

6.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
class alu_driver;
  alu_if aif (clk);
endclass
```

An interface cannot be instantiated in a class. Declare a handle: `virtual alu_if vif;`.

E2.

```
task alu_driver::send();
  @(vif.drv_cb);
  vif.drv_cb.a = 8'h10;
endtask
```

A clockvar cannot be assigned with `=`. Use `vif.drv_cb.a <= 8'h10;`.

E3.

```
class alu_driver;
  virtual alu_if.DRV vif;
  task wait_two();
    ##2;
  endtask
endclass
```

`##N` requires a default clocking, which a class does not have. Use `repeat (2) @(vif.drv_cb);`.

E4.

```
module counter_mp (cnt_mp_if.DUT cif);
  always_ff @(posedge cif.clk) cif.en <= 1'b1;
endmodule
```

`en` is an `input` in modport `DUT`; it cannot be written through this port.

E5.

```
virtual cnt_if vif = new();
```

A virtual interface is not constructed. Assign an existing instance: `vif = cif;`.

E6.

```
cnt_if cif [4] (clk);
initial
  for (int i = 0; i < 4; i++) cif[i].en = 0;
```

An array of interface instances cannot be indexed with a variable. Use constant indices, a generate loop, or an array of virtual interfaces.

E7.

```
interface io_if;
  wire [7:0] data;
endinterface

class io_driver;
  virtual io_if vif;
  task put(bit [7:0] v);
    vif.data = v;
  endtask
endclass
```

`data` is a net and cannot be assigned procedurally. Add `logic [7:0] data_drv; logic data_oe; assign data = data_oe ? data_drv : 'z;` to the interface and assign those variables.

E8.

```
task alu_driver::check();
  @(vif.drv_cb);
  if (vif.drv_cb.a == 8'd0) $display("a is zero");
endtask
```

`a` is an `output` of `drv_cb`, and an output clockvar cannot be read. Read it through a clocking block where it is an input (`mon_cb`), or declare it `inout` in the clocking block.

E9.

```
mem_if #(.AW(16), .DW(32)) mif (clk);
virtual mem_if vif;
initial vif = mif;
```

Type mismatch. The handle has the default parameters; the instance has others. Declare `virtual mem_if #(.AW(16), .DW(32)) vif;`.

E10.

```
class alu_driver;
  virtual alu_if.DRV vif;
  task idle();
    vif.valid_in = 1'b0;
  endtask
endclass
```

`valid_in` is not listed in modport `DRV`; only `drv_cb`, `clk` and `rst_n` are. Use `vif.drv_cb.valid_in <= 1'b0;`.

E11.

```
package alu_tb_pkg;
  clocking cb @(posedge clk);
    input result;
  endclocking
endpackage
```

A clocking block cannot be declared in a package (or in a class). It belongs in the interface, a module or a program.

E12.

```
module counter_mp (cnt_mp_if.DUT cif);  ...  endmodule
// ...
counter_mp dut (cif.TB);
```

The module header selects modport `DUT` and the instance selects `TB`. When both are given they must be the same modport.

Run-time bugs (the code compiles)

D1.

```
function alu_driver::new(virtual alu_if.DRV vif);
  vif = vif;
endfunction
```

The argument is assigned to itself; the class property stays `null`, and the first use of `vif` in `run()` is a fatal error. Write `this.vif = vif;`.

D2.

```
forever begin
  @(posedge vif.clk);
  if (vif.mon_cb.valid_out) record(vif.mon_cb.result);
end
```

The loop waits on the raw clock but reads clocking block samples, which may not have been refreshed yet for this edge. Wait on `@(vif.mon_cb);`.

D3.

```
task run();
  forever begin
    @(posedge vif.clk);
    vif.valid_in = 1;
    vif.a        = txn.a;
  end
endtask
```

Blocking assignments to DUT inputs at the active clock edge: a drive race. Whether the DUT sees the new values at this edge or the next depends on the simulator. Drive through the clocking block.

D4.

```
clocking mon_cb @(posedge clk);
  default input #0;
  input valid_out, result;
endclocking
```

With `#0` the monitor samples after the DUT has updated its outputs, one cycle ahead of what a synchronous receiver would see. Use `#1step`.

D5.

```
interface alu_if;
  logic clk = 0;
  always #5 clk = ~clk;
  ...
endinterface

alu_if in_if  ();
alu_if out_if ();
```

Each instance generates its own clock. The two clocks happen to be identical here, but they are separate signals with no guaranteed ordering between their edges, and they are not the DUT's clock. Generate the clock once in the top module and pass it to the interfaces as a port.

D6.

```
task read(output logic [7:0] value);
  @(negedge vif.clk);
  vif.rd = 1;
  @(negedge vif.clk);
  value = vif.data;          // X in some bits
  vif.rd = 0;
endtask
```

In the bidirectional pattern of Section 5.3, X during a read means the testbench is still driving: an earlier `write` did not set `data_oe` back to 0, so both drivers are active.

D7.

```
initial begin
  env = new(aif);
  fork env.run(); join_none
  aif.rst_n = 0;
  #100 aif.rst_n = 1;
end
```

The environment starts driving stimulus at time 0, while the DUT is still in reset, so the first transactions are lost and the scoreboard reports missing results. Apply and release reset first, or make the driver wait for `vif.rst_n` to be 1 before its first drive.

D8.

```
task drive(alu_txn t);
  vif.drv_cb.valid_in <= 1'b1;
  vif.drv_cb.a        <= t.a;
  @(vif.drv_cb);
  vif.drv_cb.valid_in <= 1'b0;
endtask
```

The task does not synchronise to the clocking event before driving. If it is called between edges, the drives are deferred to the next event, and the `@(vif.drv_cb)` that follows wakes at that same event. `valid_in <= 0` is then requested for the same event as `valid_in <= 1`; the last drive wins and `valid_in` never goes high. Start the task with `@(vif.drv_cb);`.

6.5 Coding Exercises with Solutions

Exercise 1: an interface for a FIFO

The synchronous FIFO used later in the course has the ports `clk`, `rst_n`, `wr_en`, `rd_en`, `din[7:0]`, `dout[7:0]`, `full` and `empty`. Write `fifo_if` with a driver clocking block, a monitor clocking block and modports `DUT`, `DRV` and `MON`, following the pattern of `alu_if`.

fifo_if.sv

```
interface fifo_if (input logic clk);

  logic       rst_n;
  logic       wr_en;
  logic       rd_en;
  logic [7:0] din;
  logic [7:0] dout;
  logic       full;
  logic       empty;

  clocking drv_cb @(posedge clk);
    default input #1step output #1;
    output wr_en, rd_en, din;
    input  dout, full, empty;
  endclocking

  clocking mon_cb @(posedge clk);
    default input #1step;
    input wr_en, rd_en, din, dout, full, empty;
  endclocking

  modport DUT (input clk, rst_n, wr_en, rd_en, din, output dout, full, empty);
  modport DRV (clocking drv_cb, input clk, rst_n);
  modport MON (clocking mon_cb, input clk, rst_n);

endinterface
```

The driver needs `full` and `empty` as inputs of its clocking block, because a FIFO driver must not write when the FIFO is full or read when it is empty.

Exercise 2: push and pop through a virtual interface

Write a class `fifo_driver` with tasks `push(value)` and `pop(value)` that use `fifo_if` through its `DRV` modport. Push `11`, `22`, `33` (hex) into the FIFO below, pop three values and print them.

The FIFO design (reading takes one clock: `dout` holds the value after the edge at which `rd_en` was sampled):

sync_fifo.sv

```
module sync_fifo (
  input  logic       clk,
  input  logic       rst_n,
  input  logic       wr_en,
  input  logic       rd_en,
  input  logic [7:0] din,
  output logic [7:0] dout,
  output logic       full,
  output logic       empty
);

  logic [7:0] mem [16];
  logic [3:0] wr_ptr, rd_ptr;
  logic [4:0] count;
  logic       do_wr, do_rd;

  assign full  = (count == 5'd16);
  assign empty = (count == 5'd0);
  assign do_wr = wr_en && !full;
  assign do_rd = rd_en && !empty;

  always_ff @(posedge clk)
    if (do_wr) mem[wr_ptr] <= din;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      wr_ptr <= '0;
      rd_ptr <= '0;
      count  <= '0;
      dout   <= '0;
    end
    else begin
      if (do_wr) wr_ptr <= wr_ptr + 1'b1;
      if (do_rd) begin
        dout   <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1'b1;
      end
      if      (do_wr && !do_rd) count <= count + 1'b1;
      else if (do_rd && !do_wr) count <= count - 1'b1;
    end
  end

endmodule
```

Solution. Compile `fifo_if.sv` and `sync_fifo.sv` first.

ex02_fifo_vif.sv

```
class fifo_driver;

  virtual fifo_if.DRV vif;

  function new(virtual fifo_if.DRV vif);
    this.vif = vif;
  endfunction

  task push(bit [7:0] value);
    @(vif.drv_cb);
    vif.drv_cb.wr_en <= 1'b1;
    vif.drv_cb.din   <= value;
    @(vif.drv_cb);                  // the FIFO stores the value at this edge
    vif.drv_cb.wr_en <= 1'b0;
  endtask

  task pop(output logic [7:0] value);
    @(vif.drv_cb);
    vif.drv_cb.rd_en <= 1'b1;
    @(vif.drv_cb);                  // the FIFO samples rd_en = 1 at this edge
    vif.drv_cb.rd_en <= 1'b0;
    @(vif.drv_cb);                  // dout, updated by that edge, is in the sample now
```

```
    value = vif.drv_cb.dout;
  endtask

endclass

module ex02_fifo_vif;

  logic clk = 0;
  always #5 clk = ~clk;

  fifo_if   fif (clk);
  sync_fifo dut (
    .clk   (clk),       .rst_n (fif.rst_n),
    .wr_en (fif.wr_en), .rd_en (fif.rd_en),
    .din   (fif.din),   .dout  (fif.dout),
    .full  (fif.full),  .empty (fif.empty)
  );

  fifo_driver drv;
  logic [7:0] value;

  initial begin
    fif.rst_n = 0;  fif.wr_en = 0;  fif.rd_en = 0;  fif.din = '0;
    drv = new(fif);
    repeat (2) @(negedge clk);
    fif.rst_n = 1;

    drv.push(8'h11);
    drv.push(8'h22);
    drv.push(8'h33);

    repeat (3) begin
      drv.pop(value);
      $display("[%0t] popped %h", $time, value);
    end

    @(fif.drv_cb);
    $display("[%0t] empty=%b full=%b", $time, fif.drv_cb.empty, fif.drv_cb.full);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[105] popped 11
[135] popped 22
[165] popped 33
[175] empty=1 full=0
```

Timeline: reset is released at 20. Each `push` takes two clock edges; the three values are stored at the edges at 35, 55 and 75. The first `pop` drives `rd_en` after the edge at 85, the FIFO reads at the edge at 95, and the value is in the clocking block's sample at the edge at 105. The need for the third `@(vif.drv_cb)` in `pop` is the main lesson: `dout` changes **at** the edge where `rd_en` is sampled, and a `#1step` sample taken at that edge still holds the old value.

Exercise 3: a parameterised interface and a matching virtual interface

Write a parameterised memory interface `mem_if #(AW, DW)`, a RAM that uses it, and a driver class for a memory with a 6-bit address and 16-bit data. Write two locations, read them back, and read one location that was never written.

ex03_param_if.sv

```
interface mem_if #(parameter int AW = 4, parameter int DW = 8) (input logic clk);
  logic          we;
  logic [AW-1:0] addr;
```

```
  logic [DW-1:0] wdata;
  logic [DW-1:0] rdata;
endinterface

module ram #(parameter int AW = 4, parameter int DW = 8) (mem_if mif);
  logic [DW-1:0] mem [2**AW];
  always_ff @(posedge mif.clk) begin
    if (mif.we) mem[mif.addr] <= mif.wdata;
    mif.rdata <= mem[mif.addr];
  end
endmodule

// One name for the exact interface type this driver works with
typedef virtual mem_if #(.AW(6), .DW(16)) mem_vif_t;

class mem_driver;

  mem_vif_t vif;

  function new(mem_vif_t vif);
    this.vif = vif;
  endfunction

  task write(bit [5:0] addr, bit [15:0] data);
    @(negedge vif.clk);
    vif.we    = 1;
    vif.addr  = addr;
    vif.wdata = data;
    @(negedge vif.clk);
    vif.we    = 0;
  endtask

  task read(bit [5:0] addr, output logic [15:0] data);
    @(negedge vif.clk);
    vif.addr = addr;
    @(negedge vif.clk);
    data = vif.rdata;
  endtask

endclass

module ex03_param_if;

  logic clk = 0;
  always #5 clk = ~clk;

  mem_if #(.AW(6), .DW(16)) mif (clk);
  ram    #(.AW(6), .DW(16)) dut (mif);

  mem_driver   drv;
  logic [15:0] data;

  initial begin
    mif.we = 0;
    drv = new(mif);

    drv.write(6'd40, 16'hBEEF);
    drv.write(6'd41, 16'h1234);

    drv.read(6'd40, data);  $display("[%0t] mem[40] = %h", $time, data);
    drv.read(6'd41, data);  $display("[%0t] mem[41] = %h", $time, data);
    drv.read(6'd5,  data);  $display("[%0t] mem[5]  = %h", $time, data);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[60] mem[40] = beef
[80] mem[41] = 1234
[100] mem[5]  = xxxx
```

The `typedef` gives the long virtual interface type a short name and guarantees that the property and the constructor argument have the same parameters. Location 5 was never written, and because the memory and the `data` variable are four-state, the read returns X instead of a misleading 0. This example drives on the falling edge, without a clocking block, to keep the focus on the parameters.

Exercise 4: a self-checking monitor for the ALU

Using `alu_if` and the `alu_driver` class from Section 5.2, write a class `alu_checker` that watches the ALU through the `MON` modport, computes the expected result of every operation and prints PASS or FAIL. Drive `255 + 1`, `8'h80` shifted left, NOT of `8'h0F` and `9` shifted right.

Compile in this order: `alu_pkg.sv`, `alu.sv`, `alu_if.sv`, `alu_vif_classes.sv`, then this file.

ex04_alu_checker.sv

```
import alu_pkg::*;

class alu_checker;

  virtual alu_if.MON vif;
  int unsigned pass_cnt, fail_cnt;

  function new(virtual alu_if.MON vif);
    this.vif = vif;
  endfunction

  // Reference model: the same widths as the DUT (9-bit result)
  function logic [8:0] model(logic [7:0] a, logic [7:0] b, alu_op_e op);
    case (op)
      ALU_ADD: return {1'b0, a} + {1'b0, b};
      ALU_SUB: return {1'b0, a} - {1'b0, b};
      ALU_AND: return {1'b0, a & b};
      ALU_OR : return {1'b0, a | b};
      ALU_XOR: return {1'b0, a ^ b};
      ALU_NOT: return {1'b0, ~a};
      ALU_SHL: return {a, 1'b0};
      ALU_SHR: return {2'b00, a[7:1]};
      default: return 'x;
    endcase
  endfunction

  task run();
    logic [7:0] a, b;
    alu_op_e    op;
    logic [8:0] expected;
    forever begin
      @(vif.mon_cb);
      if (vif.mon_cb.valid_out) begin
        expected = model(a, b, op);
        if (vif.mon_cb.result !== expected) begin
          fail_cnt++;
          $display("[%0t] FAIL %s a=%0d b=%0d result=%0d expected=%0d",
                   $time, op.name(), a, b, vif.mon_cb.result, expected);
        end
        else begin
          pass_cnt++;
          $display("[%0t] PASS %s a=%0d b=%0d result=%0d",
                   $time, op.name(), a, b, vif.mon_cb.result);
        end
      end
      if (vif.mon_cb.valid_in) begin
        a  = vif.mon_cb.a;
        b  = vif.mon_cb.b;
        op = vif.mon_cb.op;
      end
    end
  endtask

endclass
```

```
module ex04_alu_checker;

  logic clk = 0;
  always #5 clk = ~clk;

  alu_if aif (clk);
  alu dut (
    .clk       (clk),           .rst_n     (aif.rst_n),
    .valid_in  (aif.valid_in),  .a         (aif.a),
    .b         (aif.b),         .op        (aif.op),
    .valid_out (aif.valid_out), .result    (aif.result)
  );

  alu_driver  drv;
  alu_checker chk;

  initial begin
    aif.rst_n = 0;  aif.valid_in = 0;  aif.a = '0;  aif.b = '0;  aif.op = ALU_ADD;
    drv = new(aif);
    chk = new(aif);
    repeat (2) @(negedge clk);
    aif.rst_n = 1;

    fork
      chk.run();
    join_none

    drv.drive(8'd255, 8'd1, ALU_ADD);
    drv.drive(8'h80,  8'd0, ALU_SHL);
    drv.drive(8'h0F,  8'd0, ALU_NOT);
    drv.drive(8'd9,   8'd0, ALU_SHR);

    repeat (2) @(posedge clk);
    $display("[%0t] checker: %0d passed, %0d failed", $time, chk.pass_cnt, chk.fail_cnt);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[35] DRV: ALU_ADD a=255 b=1 accepted by the DUT
[45] PASS ALU_ADD a=255 b=1 result=256
[55] DRV: ALU_SHL a=128 b=0 accepted by the DUT
[65] PASS ALU_SHL a=128 b=0 result=256
[75] DRV: ALU_NOT a=15 b=0 accepted by the DUT
[85] PASS ALU_NOT a=15 b=0 result=240
[95] DRV: ALU_SHR a=9 b=0 accepted by the DUT
[105] PASS ALU_SHR a=9 b=0 result=4
[115] checker: 4 passed, 0 failed
```

The `DRV` lines come from the `alu_driver` class of Section 5.2. The comparison uses `!==` so that an X on `result` would be reported (Module 1, Section 4.5). This class is a monitor and a scoreboard merged into one; Module 9 separates them and connects them with a mailbox.

Further practice (no solutions given)

1. Add an `apply_reset(int unsigned cycles)` task to `alu_if` that drives `rst_n` directly, and call it from a class through the virtual interface. Why can this task not use `drv_cb`?
2. Change the output skew of `drv_cb` from `#1` to `#0` and to `#4`. Predict whether the log of `tb_alu_vif.sv` changes, then run it. What is the largest output skew that still works with a clock period of 10?
3. Change the input skew of `mon_cb` to `#0`. Predict the new log of `tb_alu_vif.sv` line by line, then run it.
4. Extend `fifo_driver::push` so that it waits while `vif.drv_cb.full` is 1 before writing. Test it by pushing 18 values into the 16-entry FIFO while a second thread pops slowly.

5. Create two ALUs with an interface array `alu_if aif [2] (clk);`, an array of two virtual interfaces and two `alu_driver` objects, and drive different operations into each.
6. Rewrite `tb_bidir.sv` so that the bus is driven and sampled through a clocking block with `inout data`.
7. Write an interface for an APB-like bus (`psel`, `penable`, `pwrite`, `paddr`, `pwdata`, `prdata`, `pready`) with modports for the requester, the completer and a monitor, and two clocking blocks.

6.6 Concepts You Must Understand Before Moving Forward

Do not start Module 9 until you can answer each of these without looking at the notes.

Interfaces

- [ ] I can explain three problems of port-by-port connections that an interface solves.
- [ ] I can write an interface with a clock port, instantiate it and connect a DUT that has ordinary ports.
- [ ] I can list what an interface may contain and the one thing it may not.
- [ ] I can explain why the clock is usually generated outside the bus interface.
- [ ] I can apply the single-driver rule to interface signals and say when a `wire` is needed.

Modports and arrays

- [ ] I can write DUT, driver and monitor modports and get the directions right from each side.
- [ ] I can state what modports enforce and two things they do not.
- [ ] I can name the three places where a modport is selected.
- [ ] I can explain why an interface array needs a constant index and how to work around it.

Virtual interfaces

- [ ] I can explain why a class cannot instantiate or directly reference an interface.
- [ ] I can declare a virtual interface, pass it through a constructor and use it to drive and sample.
- [ ] I can list four causes of a null virtual interface error.
- [ ] I can explain the three meanings of the keyword `virtual`.
- [ ] I can describe how the handle travels from the top module to the driver.

Clocking blocks

- [ ] I can describe the drive race and the sample race.
- [ ] I can list the scheduler regions in order and say what happens in Preponed, Active, NBA, Observed and Re-NBA.
- [ ] I can state the default skews and explain `#1step` versus `#0` for an input.
- [ ] I can write `@(vif.cb)`, a sample and a synchronous drive, and say when each takes effect.
- [ ] I can explain why `##N` does not work in a class and what to write instead.
- [ ] I can explain why reset is kept out of the clocking block.
- [ ] I can write the complete `alu_if` from memory.
- [ ] I can explain the pattern for driving a bidirectional signal from a class.

6.7 What Comes Next

Module 9 builds the complete layered testbench for the ALU. It uses `alu_if` exactly as defined in Section 5.1. The `alu_driver` and `alu_monitor` of Section 5.2 gain a `run()` task and mailboxes: the generator sends `alu_txn` objects to the driver through `gen2drv_mbx`, and the monitor sends observed transactions to the scoreboard through `mon2scb_mbx`. The environment creates all components and passes the virtual interface to them through their constructors, as described in Section 3.6, and `tb_top` instantiates the clock, the interface and the DUT and starts the test.